AI Guitar Tutor

A Project Report submitted

To

Manipal Academy of Higher Education

in partial fulfilment of the requirement for

the award of the degree

of

BACHELOR OF TECHNOLOGY in Data Science & Engineering

Submitted by

Rishaan Rathore

210968029

Under the guidance of

Dr. Soumya Nandan Mishra
Assistant Professor
School of Computer Engineering
MIT Manipal

Ms. Rupinder Kaur Chahal
Head of Technology
Muziclub



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

SCHOOL OF COMPUTER ENGINEERING

August 2026

DECLARATION

I, Rishaan Rathore, hereby declare that the project report titled **"AI Guitar Tutor"** submitted by me is an original record of work carried out under the guidance of **Dr. Soumya Nandan Mishra, Assistant Professor, School of Computer Engineering, M.I.T., Manipal**. The system described has been designed, implemented, and deployed by me, and all sources of information used in the preparation of this report — including the project's own GitHub repository, official technology documentation, and academic references — have been duly acknowledged.

I further declare that this report, or any part of it, has not been submitted for the award of any other degree, diploma, or similar title at this or any other institution.

Place: Manipal

Date:15 July 2026 Rishaan Rathore



_

Student Name and Signature

CERTIFICATE

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

SCHOOL OF COMPUTER ENGINEERING

Place: Manipal
Date:20/7/2026

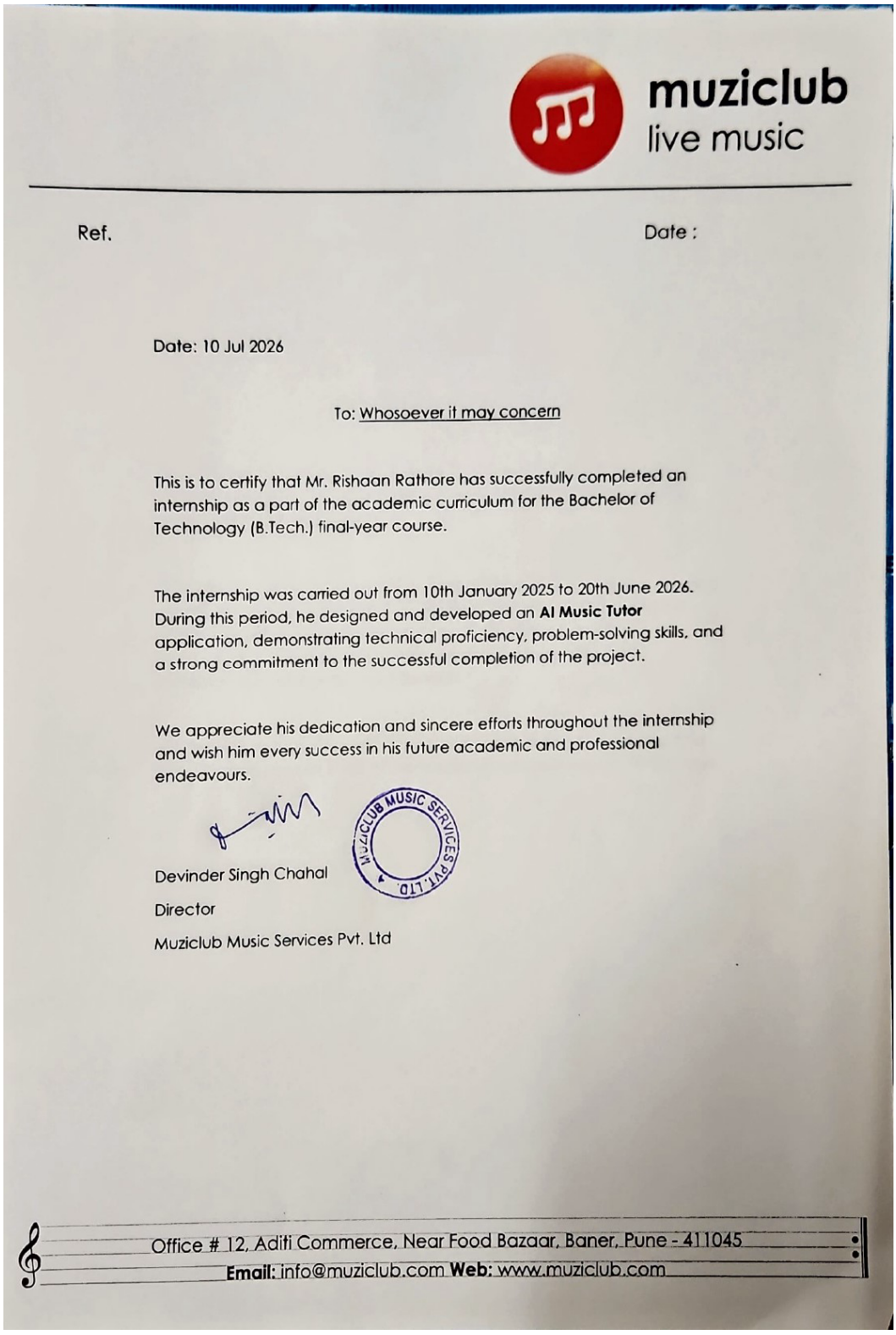
CERTIFICATE

This is to certify that the project titled **"AI Guitar Tutor"** is a record bonafide work done by Rishaan Rathore (210968029) submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in *Data Science and Engineering* of Manipal Institute of Technology, during the academic year 2025-2026.

Dr. Soumya Nandan Mishra
Assistant Professor
School of Computer Engineering,
Manipal Institute of Technology, Manipal

Dr. Smitha N. Pai
Professor & Associate Dean
School of Computer Engineering
Manipal Institute of Technology Manipal

INTERNSHIP COMPLETION LETTER



muziclub
live music

Ref. Date :

Date: 10 Jul 2026

To: Whosoever it may concern

This is to certify that Mr. Rishaan Rathore has successfully completed an internship as a part of the academic curriculum for the Bachelor of Technology (B.Tech.) final-year course.

The internship was carried out from 10th January 2025 to 20th June 2026. During this period, he designed and developed an **AI Music Tutor** application, demonstrating technical proficiency, problem-solving skills, and a strong commitment to the successful completion of the project.

We appreciate his dedication and sincere efforts throughout the internship and wish him every success in his future academic and professional endeavours.

MUZICLUB MUSIC SERVICES PVT. LTD.

Devinder Singh Chahal

Director

Muziclub Music Services Pvt. Ltd

Office # 12, Aditi Commerce, Near Food Bazaar, Baner, Pune - 411045

Email: info@muziclub.com **Web:** www.muziclub.com

ACKNOWLEDGEMENT

I wish to express my sincere gratitude to all those who have supported and guided me throughout the duration of this final year project. I extend my heartfelt thanks to the **Dean of School of Computer Engineering,Dr. Radhika M. Pai for** providing the necessary infrastructure and a conducive environment for research and project work. My sincere appreciation goes to the **Head of the Department of School of Computer Engineering, Dr. Smitha N. Pai** for their encouragement and for making the resources of the department available.I am profoundly grateful to my **Project Guide, Dr. Soumya Nandan Mishra, Asst. Professor School of Computer Engineering**, for their invaluable guidance, insightful suggestions, and constant supervision. Their expertise and unwavering support was instrumental in the successful completion of this project. I also thank Mr. Devinder Singh Chahal, Director, Muziclub Music Services Pvt. Ltd., for hosting and supporting the industry internship (10 January 2025 – 20 June 2026) during which this application was designed and developed.

ABSTRACT

For most people, first encountering a guitar is essentially like being thrown in the deep end. You strum a chord, something sounds... Wrong, in a way you can't quite articulate, and then you're immediately stumped. It's finger placement? A dead string? Is this just beyond you? Without another person sitting there, someone to explain everything plainly, it’s impossible to know. AI Guitar Tutor aims to bridge this gap. It's a full-stack web app that combines a microphone-based chordchecker with LLM-generated lesson plans; the whole package fits into a unified practice workflow. On a technical level, it's a Next.js 14 frontend hosted on Vercel, a FastAPI backend running on Google Cloud Run, and a PostgreSQL database handling all the heavy lifting in the background.

The architecture has two main engineering challenges; the first is an audio evaluation engine. This is a bit more complicated than it sounds. A small audio snippet is recorded by the user directly through their browser's microphone, then passed to be processed by librosa and scipy. Next, it runs through a combined-chromagram-and-FFT pipeline, utilizing adaptive gap-based thresholding, semitone deduplication, and harmonic-series rejection, to identify the notes actually played that belong to the desired chord. The resulting assessment is much more than just a simple "correct" or "try again" -- it gives a quantitative score along with a detailed analysis, highlighting which strings were played correctly, were omitted, or were muted.

The second engineering challenge-and this one was arguably much more experimental-is a fouragent lesson generator that I've been referring to as the Council. A user provides a song name, and a Llama 3.3 70B model running on Groq breaks the song down into four discrete, niche tasks: music theory, technique, ear training, and the practice plan itself. Each one of these tasks runs somewhat independently, then a fifth agent, or the Chairman, ties it all together into a single coherent document-including chord fingerings and a remedial quiz at the end. It’s a bit like having four specialist tutors have an argument, with the Head Tutor taking minutes on the outcome.

Layered on top of these two systems is a more traditional set of practice-related features, though none should be considered merely boilerplate: an interface for practicing individual chords with SVG fretboard feedback showing ideal positions versus actual positions; a chord transition trainer that time how quickly you canswitch between chords (another big tripping point for most beginners); a personalized lesson planner that develops a curriculum based on your past performance; and a progress dashboard featuring a 52-week activity calendar and a chord mastery matrix. The whole system is secured by JWT authentication with refresh-token rotation, so security was never an afterthought.

This report will outline the architecture, delve into the implementation details behind the audio evaluation engine and the LLM lesson generator (instead of just their theoretical foundations), describe the data model, and detail the end-user workflow. It will also take a more in-depth look at how well the system's implementation meets its original aims, where it has fallen short, and potential next steps that could prove beneficial.

Table of Contents

DECLARATION ***2***

ACKNOWLEDGEMENT ***5***

ABSTRACT ***6***

LIST OF FIGURES ***9***

LIST OF TABLES *9*
1.1 Preamble **10**
1.2 Background **10**
1.3 Problem Statement **10**
1.4 Existing Challenges **11**
1.5 Motivation **11**
1.6 Objectives **12**
1.7 Scope **12**
1.8 Contributions **13**
1.9 Significance **13**
2.1 AI in Education and Intelligent Tutoring Systems **15**
2.2 Music Information Retrieval **15**
2.3 Digital Audio, Sampling and the Short-Time Fourier Transform **16**
2.4 Pitch Detection and Chroma-Based Chord Recognition **16**
2.5 Large Language Models and Multi-Agent Decomposition **17**
2.6 The Web Audio API and Browser-Based Recording **17**
2.7 Literature Review and Related Systems **17**
2.8 Research Gap **18**
3.1 Development Methodology **19**
3.2 High-Level System Architecture **19**
3.3 Technology Stack **21**
3.4 Repository and Folder Structure **22**
3.5 Frontend Architecture **22**
3.6 Backend Architecture **23**
3.7 Database Design **24**
3.8 Authentication Flow **24**
3.9 Audio Evaluation Pipeline **25**
3.9.1 Pipeline overview 25
3.9.2 Dual detection: chromagram and FFT 25
3.9.3 De-duplication and false-positive rejection 26
3.9.4 Scoring 26
3.10 Council: AI Lesson Generation Pipeline **26**
3.11 API Design **28**
3.12 Deployment Workflow **29**
3.13 Testing Strategy ***30***
4.1 Landing and Chord Practice Interface ***31***
4.2 Authentication Screens ***33***
4.3 Song Council and the Integrated Lesson View ***35***
4.4 Transition Trainer **38**
4.5 Dashboard and Personalised Daily Plan **39**

4.6 Progress Analytics 40
4.7 Result Discussion and Limitations 41
5.1 Summary 43
5.2 Achievements 43
5.3 Limitations 43
5.4 Future Scope 44
5.5 Final Conclusion 44
REFERENCES 45
APPENDIX A — PROJECT FOLDER STRUCTURE 46

LIST OF FIGURES

Figure 3.1 High-Level System Architecture 12

Figure 3.2 Audio Evaluation Pipeline 19

Figure 3.3 Council Lesson-Generation Pipeline 20

Figure 3.4 Authentication and Token Refresh Sequence 18

Figure 4.1 Landing Page and Chord Selector 23

Figure 4.2 Guided Practice Flow, Recording and Fretboard Feedback 24

Figure 4.3 Login Screen 24

Figure 4.4 Sign-Up Screen 25

Figure 4.5 Song Search and Council-Generated Lesson Summary 26

Figure 4.6 Council Lesson-Generation Loading State 27

Figure 4.7 Transition Trainer Drill 27

Figure 4.8 Dashboard: Today's Plan and Quick Stats 28

Figure 4.9 Progress Dashboard: Activity Heatmap and Skill Meter 29

LIST OF TABLES

Table 3.1 Frontend Technology Stack 13

Table 3.2 Backend Technology Stack 13

Table 3.3 Infrastructure and Hosting 14

Table 3.4 Database Schema Overview 17

Table 3.5 Core and Auth API Endpoints 21

Table 3.6 Council, Progress and Personalisation Endpoints 22

Table 3.7 Automated Test Coverage 22

Table 4.1 Screen-to-Module Mapping 30

Table 4.2 Implemented vs. Documented-Only Capabilities 30

CHAPTER 1

Introduction

1.1 Preamble

Here in this chapter, let me introduce what AI Guitar Tutor is: a web application that produces human-sounding audio-response on your chording accompanied by custom lessons provided by the AI which is targeted at the song that you are going to learn. But first, in order to demonstrate why such application is needed, let me introduce the problem that is posed by self-learning of guitar. I will then describe where are the weaknesses on existing resources that facilitate self-learning. Finally, I will introduce what is the main motivation for the project, as well as what is in the scope, and what value of the thesis in term of knowledge and academic contribution.

1.2 Background

What generally happens when somebody is learning how to play guitar: It consists, to most people's experience, of the exact same few things - a chart of chords open on someone's telephone, some tab site, a couple of video with someone painstakingly guiding finger position, after which you're simply repeating stuff silently (again and again), expecting it to penetrate. The problem usually is that none of things ever allow you to understand whether what you actually have to play actually was the right factor. An open chart of chords may make you understand where you stand meant to move you're on a finger, yet it generally does not know if that open A string you have already been even remotely mindful in the present simply rang out as well as destroyed the complete chord. And video courses (even though perfectly good any) only speak toward someone.

This cannot listen back again. The core idea it boils to is: There are few instructional stuff on a number of topics. You can drown from instructional films for guitar over YouTube, and that I would even venture to express: it doesn't go significantly until the teaching is free. The disparity connecting so much instruction as well as practically nil correction are exactly what led me to develop some type of audio-aware thing..

1.3 Problem Statement

Imagine you are a beginner practicing a guitar chord chart without an teacher present. You strum the G major chord, but it sounds a little dirty, and … what now? You honestly don't know if that dirtiness comes from one of your fingers resting against the open high E, if you are not pressing hard enough on the third fret, or if it is something totally different.

You really have no real way of checking.

So you basically just take a guess, strum again and repeat your guess for, not ideal if the hope is to teach yourself a new chord correctly from the get-go. Then there's another second problem which has the odor of it all, although it really takes a completely separate route! Say you want to discover

a particular chord progression. This really is something your typical beginners usually strive to discover-may be some in Ed Sheeran's repertoire, or a timeless folk melody perhaps originating from wedding.

What if nobody has already created training videos for thatparticular song (a good one at that)?

You are up to chance and go searching on the internet, hopefully luck being on your side,or you spend hours yourself trying to figure out the exact chord structure and overall progression. Given the common lack of knowledge in the area of music theory, few Beginners can carry out this "reverse-engineering" alone, which means these efforts seldom bear fruitful outcomes. These were two issues that drove the construction ofAI Guitar Tutor really, both seeming far too intimately linked to effectively resolve individually.

In one place, there's chord scoring which evaluates, based on actual acoustic indications rather than mere speculation. And in another area is automatic instruction generation that responds directly to your choice of song and can be done for any tune you throw at it. We'll review later whether these together make everything perfect, but that was the rationale behind grouping everything under one roof.

1.4 Existing Challenges

Building a system that solves this problem in a browser, without dedicated hardware, surfaces several concrete engineering challenges that this project had to resolve:

- **Noisy, an uncontrolled input.** Audio arrives as a short microphone recording of unknown quality, with room noise, off-axis pickup, and no guaranteed silence before or after the stums
- **Distinguishing the "wrong chord" from the "right chord, or poorly played."** A muted string, an extra open string, and a completely different chord all need to be classified differently so the feedback is actionable rather than a single pass/fail signal.
- **Cost and latency of AI-generated content.** Producing a full song lesson from a single largelanguage-model call is slow and prone to shallow might lead to hallucination , generic output; producing it from several models run independently multiplies latency and API cost.
- **Personalisation without a heavyweight model.** A meaningful "what should I practise today" recommendation has to be derived from a student's own attempt history rather than a trained recommender system, given the project's scope and data volume.

1.5 Motivation

Honestly, some of the motivation for doing this was kind of a personal experiment. Could somebody, like, actually pull off doing something like this themselves: some old-school DSP, together with a new-fangled LLM API call, get it working, end up with something useful, not just, like, cool tech demo stuff. That was kind of where it started.

And when you have an LLM lying around, the temptation is to, like, have it doing everything, right.

You can have it describing what a chord sounds like, having it give you feedback, whatever it may be. And that feels a little empty, to me. Because if you want something to actually give you feedback, that feedback should come from hearing what you just played. And not, like, having your language model trying to imagine what feedback feels like or what kind of feedback would be given on that.

So the acoustical component has to be real.

It has to have some kind of signal analysis underpinning it, and not the text-generating kind that's trying to be analytical. And then there's a second layer of sort of grounding there, the lessons the AI generates aren't just arbitrarily disconnected from the rest of what's happening in the application. They leverage the same chord and fingerings data that is being used by the acoustical engine.

So when the council suggests a lesson about, you know, an open D chord, it's drawing on the same kind of data, the same kind of fundamental structure that the score checks against the audio stream itself. Now, I don't know if the average user cares one way or the other about that, but architecturally it felt right.

1.6 Objectives

---

1. Designing and implement an audio evaluation engine that scores a recorded chord attempt against a target chord and identifies specific error types (missing note, muted string, extra note, etc.).
2. Designing and implement a multi-agent AI pipeline that turns a user-supplied song name into a structured, multi-section lesson (theory, technique, ear training, practice plan, quiz) grounded in that song's actual chord vocabulary.
3. Build a practice interface chord selection, microphone recording, SVG fretboard feedback, and a timed transition drill around these two engines.
4. Persist practice history per authenticated user and deriving a personalised daily practice plan and progress analytics from it.
5. Deploy the system as a publicly reachable web application with continuous deployment from source control.

1.7 Scope

I think it's important to be really clear about what this system actually does versus what it might sound like like it does. So the audio analysis is only going to be working on single chords, in individual stabs -- one recording of one strum being analyzed at a time. It's not continually watching you as you play through a piece; that would be a meaningfully different (and much harder) engineering challenge than we're taking on in this project.

Similarly, the chord-selection mechanism of generating song-lesson is purely reliant on whatever the language model happens to know already about that specific song in the existing corpus it was trained on; it's not pulling that information from audio input for a particular song file.

If you decide to try it out, don't expect that the lesson you request for, say, 'that popular folk song of questionable copyright status', or 'that piece I just wrote yesterday morning' are going to produce incredibly satisfying or accurate results. These are obvious points to make before the end, and they should ideally sit around with the reader, not get dismissed as footnotes. At its center, the vocabulary for what we're practicing here consists of 84 human author-constructed chord-based lessons, each with existing recorded reference audio and fingering info. These lessons form the foundation.

Just to be on record what's decidedly out of scope for this project, I should note there's the matter of using neural pitch detection rather than just using classical signal-processing, but I tried to make sure my limitations were listed openly.

In the interest of full disclosure, you probably don't really want to think too hard, yet, about the lack of a nice native mobile app; or how the system would actually respond to your strumming through an entire piece, live; but they should show up later, in chapter 5, and I hope I didn't sneak them in there too stealthily.

1.8 Contributions

In terms of what the project actually provides, I'd classify it as having three real, tangible contributions worth listing out.

1. The chord evaluator, at the heart of everything: Based on a combination of both chromagrams and the FFT, the decision on whether a note has "sounded" is not an absolute, fixed-threshold based on loudness but relative to the signal itself; this flexibility is crucial when it comes to accommodating both room acoustic noise and varying microphones used by different people.
2. The lesson-generation pipeline itself consists of four specialized, role-playing, generative agents and a chairman, with each of their specific prompts focusing their output rather than posing as much as possible for a single query against one LM. From a practical point of view, these end up running two rate limited pairs (to try and keep costs within reason within a free/low-tier API service). This isn't an ideal constraint, but it is a very real one that constrained the solution design.
3. The personalization heuristic itself does what it suggests: It selects a practice plan directly from a student's past practice attempt log, not by calling an additional trained model but by directly looking into what the student has demonstrated difficulty with previously.

Each of these components are also not stand-alone or de-linked in any way - the application is designed as an end to end, working application rather than simply three individual demos residing in the same file directory. That differentiation to me seemed important to explicitly mention; it would have been far easier, and I would argue, far less practical. To just develop three prototypes.

1.9 Significance

I think there's something here that's slightly more than a class project. For me, the system really settles into a useful pattern for beyond guitar-related applications: pairing a deterministic DSP pipeline and a generative AI layer but distinguishing deliberately who's in charge of what. The AI is the planner and expositor; the AI writes the lessons, deconstructs theory, formulates a practice session that's read with flow and is actually coherent to the new guitarist.

It will, however, not judge on your ability to play the correctly held chord.

The signal processing layer makes the real determination based on physical, acoustic data; it won't hallucinize what a proper G chord is. TheDSP system is the ground truth; the language model is merely the narrator. You can tell yourself it matters a great bit: Had you given a language model " does this sound like the right version of this chord" perhaps one could've done it, and said a prayer, but ultimately your app is going to make an awful lot of wrong assertions and mislead people and betray their confidence. It is a very sound idea which can apply to an array of products that mix sensordata with generated contents.

CHAPTER 2

Background Theory

2.1 AI in Education and Intelligent Tutoring Systems

ITS - smart or not “ITS are systems in the tradition of the field of Intelligent Tutoring Systems, colloquially often short for just IT “ITS” - a system that is, you will see below, supposed to adapt and tailor what and how it teachies to you given who you are as learner. Where the adaptiveness comes from has been on the back end historically, though - based on rule-based Student Models, or, manually developed student profiles that were derived by a human “authoring” the system as “if student cannot master this fact, present the following explanation.” As noted above, however this would mean that an ITS would need to either learn that explanation and content “off the fly” by an algorithm as well; on the flip side of the ledger, the work of an LLM has removed much of the heavy manually authoring a substantial library of explanation-and-associated-practice – instead, it is able to respond instantaneously to requests, at request time by-the-by, with an explanation, explanation and practice or practice plan tailored for pretty-much any skill, any knowledge point.

It does feel though as, even if an “AI model” does do some of the work as I indicated above, it does not do anything, by virtue of where and how it lands it on my side of the screen, do much on back end by way of rule-based explanations and checks.

Except I am not a fan of this way of design. Because AI Guitar Tutor places some pieces of its operations squarely in this generation-at-request-time, less than deterministically explicable category. I discuss the “Council system” properly, including some of its merits, or demerits “The Council system which I cover more broadly in Section 3.10”, is the first point in our discussion above- “and how it generates theory, and technique, and ear-training content at request time rather than by referencing a library “on” a pre authored basis. The chord classifier though; the personalization plan etc are more old style.

Deterministically traceable to human thought.

2.2 Music Information Retrieval

That's because Music Information Retrieval, or MIR, is what these projects are all involved in, that is to say getting musically interesting facts out of an arbitrary piece of audio: the pitch, the chord, the key, the tempo, how the overall piece is structured. And chord-detection is itself a particular component within MIR research. This piece deals with nothing but chord detection although in a very much more basic form than one has come to associate the term 'MIR research.'

However, the conventional practice, adopted here, of taking a short chunk of audio, and transcribing that in some sense to a 12-pitch space - we usually called it a 'chroma-vector,' and I'll adopt that-is to simply compare it to all the template representations of those chords one may care about and assign the one that matches best. Most advanced systems do try to perform true classification, that

is, trying to determine which chord from all possible ones best represents this given fragment of audio. It's genuinely a hard classification problem.

My audio system - which I cover in quite detail in chapter 3.9 - uses exactly the same strategy but on a somewhat smaller scale, where it's not trying to figure out from an arbitrary set of chords exactly which chord someone has played; in fact it knows exactly which chord you tried to play, because that's the one which the lesson asked you to play; so on one level it's really verification - we simply want to see which ones of that given chord-whose pitch-classes are known and have we observed on the signal. So again, it's a much simplified problem, perhaps - and I hope more useful one on our terms - as there's only one candidate of interest.

2.3 Digital Audio, Sampling and the Short-Time Fourier Transform

---

So a microphone represents a continuous input signal and naturally, a computer knows nothing of continuous. It can store a good discrete sampled equivalent which, for the purpose of reconstituting the original from snapshots made close together, is “close enough.” A point worth stopping for briefly to appreciate.
Everything incoming is resampled to 22.05 kHz, mono. Again not just picked a number because it seems so - 22.05kHz is a common sample rate used forMIR, picked because at this point the signal is capable of retaining all of the important, the Fundamental and first harmonics of the string sounds we're likely to find for any of the strings on a guitar but not so data intensive as to take excessive computing resources to analyze the data.
In order to work out which frequency has come into the system you can't look directly at a single waveform cycle - the information that matters for this is in none of the individual cycles themselves. This results in the overall wave being broken up into overlapping sections called 'frames.' This involves deciding on both a 'framing size' (how large each slice) and the 'hop length' (how far apart they start). And each of those chunks then going through the process that figures out the amount of energy in each frequency bin for that fragment of wave – otherwise known as the 'Fast Fourier transform', often referred to simply as the 'FFT'.
The output of this process is actually the 'magnitude spectrum' of the framing – an estimate of energy in each individual frequency bin contained within each particular framing. And it is those multiple framings, shown back-to-back (over time) which produce what we know as the 'spectogram', effectively a graph plotting time and frequency - this is the raw material an FFT based 'detector' (like the engine's one) works with.

.

2.4 Pitch Detection and Chroma-Based Chord Recognition

For our evaluative task relevant to this project, we employ a combination of two complementary techniques. The first is peak-picking on the FFT magnitude spectrum : any local peak above some minimum amplitude and prominence threshold is assumed to be the fundamental pitch (thus considered as candidate note). This is converted to the nearest musical note based on the equally tempered note to frequency equation: note = 12 log(f / fref), taking A4 = 440 Hz as the reference pitchfref .

Secondly, there exists the “chromagram”, more efficiently calculated from a Constant-Q Transform (CQT) than a simple linear FFT.

The CQT compresses multiple octave copies of the same “pitch class” into one single bin in the spectrum - for example, both E2, E3, and E4 might result in energy in the same E-shaped bin. Because a chord contains pitch energy over many octaves due to each strummed note on a different guitar string spanning multiple octaves, a chroma feature computed from a CQT will be much less susceptible to noise than a simple FFT peak detector finding only the energy at the note closest to the true fundamental on that one specific chord string would. Both are run separately for our system’s judge/evaluator and must find agreement (or other corroborative evidence), as discussed more in Section 3.9 .

2.5 Large Language Models and Multi-Agent Decomposition

If you ask a language model to just “write a full lesson” in one massive prompt, what usually shows up is wide but shallow. It sort of covers the map but really only superficially, because the model has to distribute its output thin across a ton of loosely connected sub-topics, in one go. Practice, technique, theory, ear training, it’s all jumbled in there to an equivalent word budget.
Another approach for something like this is to use the multi-agent pattern, or perhaps the “council” pattern which I think is actually a fitting descriptor. Rather than stuffing all the prompt details into one giant query, you divide the work into distinct, tighter prompts for each specialized agent. That way, each can be given tailored system prompts and only those parts of the input that are relevant to its specific role. And after running independently, a final assembly pass puts it all back together. Our Council pattern, described in detail in section 3.10, is simply a particular instance of that. It consists of 4 specialist agents - for practice, theory, technique, and ear training - each of whom receives access only to the inputs necessary to fulfill its specialized function. We then have each specialist produce just a short, contained output, about 200-250 words maximum. Finally, a fifth Chairman agent takes all the outputs and integrates them into a single lesson document, rather than just a messy collage of separate outputs.

2.6 The Web Audio API and Browser-Based Recording

Getting microphone audio out of a browser tab, no native app, no plugin, relies on two browser APIs working together. First there's MediaDevices.getUserMedia(), which handles the actual permission request, that little popup asking if the site can use your mic. Once that's granted, MediaRecorder takes over and encodes the resulting audio stream into an actual file, typically WebM with Opus encoding, though it depends a bit on which browser you're using, sometimes you get WAV instead. That file gets packaged as a blob and shipped off to the server.

This project's frontend leans entirely on this browser-native recording path. No plugin, no native shell wrapping things. Which sounds like a small implementation detail, maybe, but it's actually what makes the whole practice flow, record, submit, get a score back, work as an ordinary web page that anyone can just open in a browser tab without installing anything first.

2.7 Literature Review and Related Systems

Broadly speaking, when I looked at the existing landscape of guitar-learning software, especially relating to this particular project's considerations, it seemed to fall into three buckets. There is static referential information: the kind of thing you'd find on tab or chord-diagram sites. Helpful, of course, and something many players utilize at some point.

However, there is absolutely no audio response.

It tell us that “here is how you finger this chord,” and implicitly assumes we have. There is, then, audio input practice: a tool which attempts to actually, acoustically determine whether our fingers have landed the appropriate pitches, via a note or chord recognition system, to detect success or failure in either the moment it occurs, or post-performance. Typically, however, what this category has is the problem of content, generally being, of necessity, laser-focused on the detection problem rather than what would, for most of us, be the more important challenge – something like… guided learning on a particular song that I actually desire to learn. Now, there is a very new and emerging third bucket of practice, which, again, because we have Large Language Models, has the capability of content generation for, essentially, any topic a user could possibly imagine wanting learning materials for – quizzes, exercise plans, explanations.

The limitation of the third bucket seems obvious: for the vast majority of it, there's absolutely zero connection to any kind of acoustic verification, we’re just making stuff up, not checking anything.

Given that landscape, what the current project is adding is not so much a new category of software per se, as it’s connecting two that typically don't combine to great effect in one workflow: that is, combining audio input practice of song-related work, with AI generated content for a song, to have both, as much as possible, in one user experience as opposed to juggling two disparate applications. Whether that, combination provides a genuinely superior experience for a learner than using the two best in breed individual pieces of software individually, is again a fair question, and I think it may well depend on the learner. But it can at least save me the trouble.

2.8 Research Gap

The specific gap that this project is designed to fill, more specific even than the last category, is the one directly in the middle that is between those two categories I just explained. Chord detection tools simply give confirmation that something was played, they tell you very little else. AI-content tools can teach, and oftenteach very well, but have no mechanism of verifying if something was done correctly and in this case: whether you learned from the material they generated.
It's more than just that the two exist in separate applications as well though. It's common that audiopractice tools will send the detection results into some form of curriculum, but often there's very little structured or personalized feedback beyond a 'correct' and 'incorrect', or the same exercise endlessly repeated without connecting the session, or over time. Correspondingly, AI-tutoring content tools typically don't anchor the content they generate based on what data the learning is actually being evaluated against; a lesson might reference some chord shape that doesn't fully align with what the audio engine anticipates and this wouldn't be caught.
The Council system of this project attempts to close the feedback loop between these things more directly than the current state: instead of being perceived as limitation of the current applications, the same files Fingerings.json, Chord_definitions.json that score the user's input can be passed into

the AI content tool; this is explained more in Section 3.10. In effect, this assures that when content is generated by the AI, it will not include chord shapes that the audio engine does not know how to assess, such a minor detail, perhaps, but it prevents both sides of the application from drifting away from each other.

CHAPTER 3 System Design and Methodology

3.1 Development Methodology

The way we developed this wasn't this big waterfall approach in which everything was envisioned, planned, then built step by step. It was developed in increments, built in bits and then individually somewhat validated, then we built on it. Our audio evaluating component is an increment that was built and tested by itself, then our Authentication and Data tier, then the Council AI pipeline, and on top of all that, our Personalization and Progress Tracking layer.

So no fixed single document that the project specification is…if you could even say we have a document…the thing that was the specification was the set of test cases in the repository (see "Testing") which also defines the document .archive.md andaudio-algorithm.md (design doc's which evolved as we evolved).

So again, a good case in point, is the current formula in evaluator.py is not the first formula we spec'd out - we spec'd out an initial weighted average formula that was pretty nice on paper. The one we ended up with is a simplified empirical adjustment. (See "Scoring" section ["3.9.4 Scoring"] for detail - the short and simple explanation however, that what testing pushed us toward.) Which is the way it ought to be developing when you go incrementally as opposed to trying to pretend you could ever perfectly specify your solution out first.

3.2 High-Level System Architecture

My system splits nicely into two applications that can be deployed independently of each other and communicate with one another only ever via REST. Not much can be tighter than that. My frontend is writing it up using Next.js, running it there.

The backend is written up with FastAPI, running it somewhere else, such as off Render, such as all run as an additional component in the diagram below.

Both front end and back end will consult as their single persisting entity the same fully-managed instance of PostgreSQL. BelowFigure 3.1is diagramming exactly all that and a bag of bits.

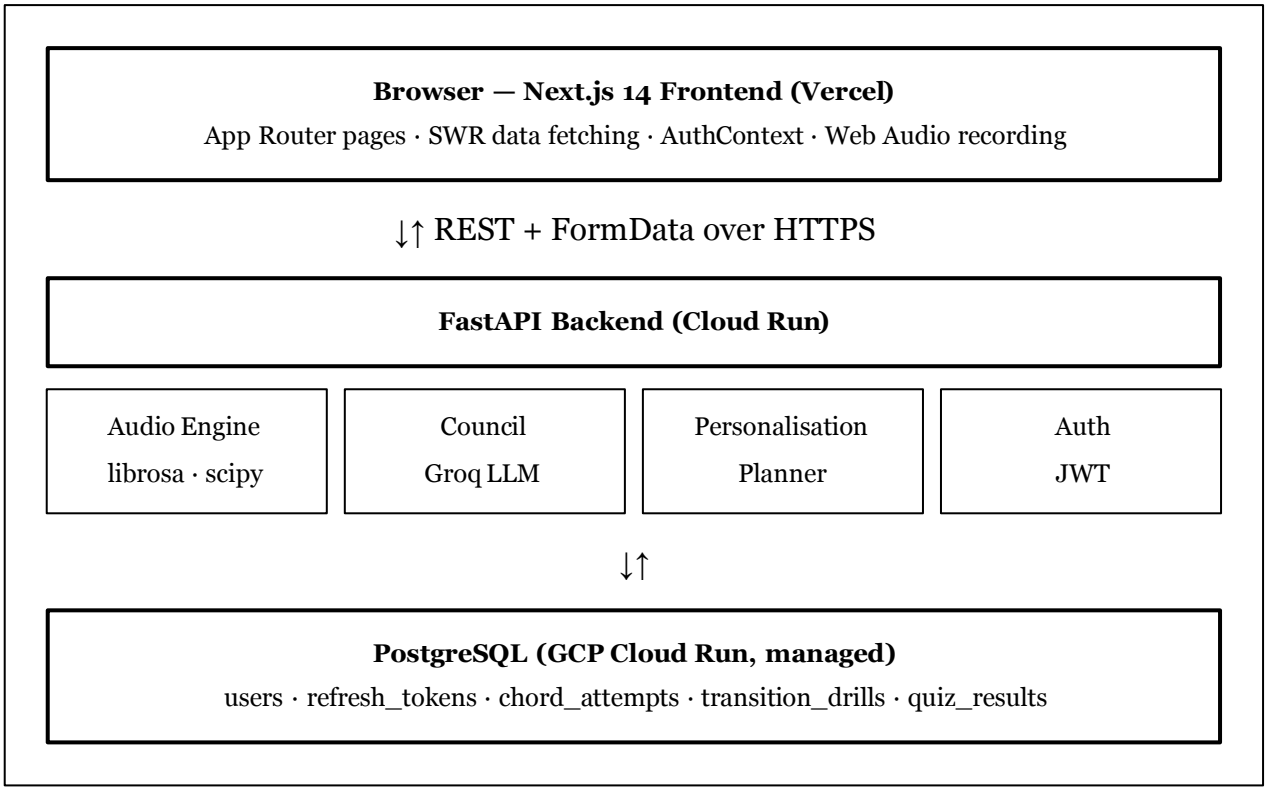


Figure 3.1. *High-level system architecture: the frontend and backend are fully decoupled and communicate only through REST; both auto-deploy from the same GitHub main branch.*

3.3 Technology Stack

Table 3.1 and Table 3.2 list the concrete libraries in use, drawn directly from frontend/package.json and backend/requirements.txt.

Package	Role
Next.js 14 (App Router)	Framework: routing, rendering, build/deploy target
React 18	Component model and rendering
TypeScript	Static typing across components and API layer
Tailwind CSS	Utility-first styling
shadcn/ui + Radix UI	Accessible component primitives
Framer Motion	Page transitions and card animation
SWR	Remote data fetching with caching/revalidation
Lucide React	Icon set
Sonner	Toast notifications
cmdk	Command palette

Table 3.1. *Frontend technology stack.*

Package	Role
FastAPI	Async REST API framework
Groq (llama-3.3-70b)	Hosted LLM inference for Council agents and feedback tips
librosa, scipy, numpy	Audio loading, signal processing, pitch/chroma detection
SQLAlchemy 2.0 (async)	ORM over the data layer
asyncpg / aiosqlite	Async Postgres driver (prod) / SQLite driver (local dev fallback)
Alembic	Database schema migrations
python-jose, passlib[bcrypt]	JWT issuing/validation and password hashing

Table 3.2. *Backend technology stack.*

Service	Role

Vercel	Frontend hosting; auto-deploys on push to main
GCP Cloud Run	Backend hosting; auto-deploys on push to main
GCP Cloud SQL	Managed production database PostgressSQL
GitHub	Single source of truth for both deploy targets
Secret Manager	Stores `JWT_SECRET_KEY`, `GROQ_API_KEY`, `DATABASE_URL` |
Artifact Registry	Stores built backend Docker images
Cloud Build	Builds backend container image
Cloud Storage	Public bucket serving lesson video/audio assets (`gs://aiguitar-tutor-lessons`) |

Table 3.3. *Infrastructure and hosting.*

3.4 Repository and Folder Structure

The repository structure is of a monorepo, which cleanly delineates its frontend, backend, and data aspects; a brief overview of the same is provided below while a complete structure can be viewed in Appendix A.

```
ai-guitar-tutor/
├── frontend/src/app/          Next.js App Router pages (/, /login, /signup,
/dashboard, ...)
├── frontend/src/components/   SongCouncil/, feedback/, layout/, ui/
├── frontend/src/lib/          api.ts, auth.ts, motion.ts, progress.ts,
noteDetection.ts
├── backend/audio_engine/      ChordEvaluator (librosa + scipy)
├── backend/council/           4-agent lesson generation system
├── backend/feedback_engine/   tip + fingering guidance generator
├── backend/auth/, db/         JWT auth, SQLAlchemy models, async engine
├── backend/progress/, personalization/
└── data/lessons/ (84 chord folders), data/chords/ (fingerings.json,
chord_definations.json)
```

3.5 Frontend Architecture

The frontend is built using the Next.js 14 App Router and will use “use client” on interactive components with layout’s and base shells stay a Server Component. We’ve built table-driven routing where each of the top-level paths resolves to one page component, see the table above for all top-level routes.

Route	Purpose	Auth
/	Landing page and chord practice interface	Optional
/login, /signup	Authentication	No
/dashboard	Today's plan, quick stats, streak heatmap	Required
/progress, /progress/chord/[name]	Progress analytics and per-chord drill-down	Required
/profile	Display name and skill-level settings	Required

We decided on 3 independent sources for state management at this layer rather than attempting to use one large global state store to manage it all (a decision that may very well have resulted in more complex spaghetti code!). First, there is a React Context object which defines the global auth state, or login status and session information which we are calling the AuthContext. There is another state management layer which stores all remote state via the SWR hooks, where each hook is identified by its path to the resource.

Lastly, there is the truly local UI state, e.g.

Whether a dropdown menu is open, or which tab is currently selected, in which we are using component-level useState hooks to manage. We highlight one more thing at this layer, and that is each chord-practice request also has a X-Session-Id header set by the client to crypto.randomUUID(); The reason this is required is that the back end is effectively stateless, and by themselves don't have the means to recognize and route subsequent calls to the same online instance.

3.6 Backend Architecture

The sole FastAPI entry point isbackend/main.py. At startup it one-shot instantiates the statelessLessonService,ChordEvaluator andFeedbackGeneratordepending on them from requests. It also loadsthe fingerings.jsonconfiguration file, initialisesCORSdepending on theALLOWED_ORIGINSenvironment variable,and then mountsthe Councilrouter,authorizationrouter,progressrouterandpersonalityrouter. Practice session data is stored in amemorydictionary keyed bysessionid, which is subject to a onehourTTLandopportunistically garbagedged on each request-this is an intended design decision (seeTable), as practice sessions are inherently transient and lack need for database persistence.

Each active session owns one TutorAgent instance (tutor_agent/agent.py), a small state machine:

```
IDLE → TEACHING → WAITING_FOR_PLAY → ANALYZING → FEEDBACK → (WAITING_FOR_PLAY or COMPLETED)
```

A selection of the project's own documented architectural decisions, with their stated rationale, is reproduced in the table below.

Decision	Rationale
4 parallel/paired LLM agents instead of 1	Shorter, focused prompts produce higher-quality sections than one large prompt
Fingerings embedded into the lesson at generation time	Removes a separate fingering round-trip during practice
SQLite fallback for local development	Same async SQLAlchemy code runs against SQLite or Postgres with no Docker dependency
Session state kept in memory, not the database	Practice sessions are ephemeral; DB persistence would add latency with no benefit
X-Session-Id header instead of cookies	Lets a stateless backend route to the correct TutorAgent without cookie/session infrastructure

3.7 Database Design

We model persist data with SQLAlchemy 2.0 async models. Migrated by Alembic and store in prod database using Render's managed PostgreSQL (defaults to SQLite using aiosqlite if DATABASE_URL environmental variable is not set.)

Table	Purpose
users	Account record: id, email, password hash, display name, role, skill level
refresh_tokens	Hashed JWT refresh tokens, supporting rotation
chord_attempts	Every audio submission — chord, score, detected/missing/extra notes, feedback text
transition_drills	Every chord-to-chord transition drill result (TPM, hit/miss counts)
quiz_results	Pass/fail record per lesson quiz

Table 3.4. *Database schema overview (source: backend/db/models.py).*

3.8 Authentication Flow

Auth scheme uses short-lived JWT access tokens (15 minute expiration) and long-lived refresh tokens (30 day expiration, refreshed tokens are hashed, not clear-text, so that a breach of the DB wouldn't grant reuse) and require at least one of 3 roles (student, teacher, admin) a FastAPI depend using a function such as require_role(role), all hashed with passlib (bcrypt).

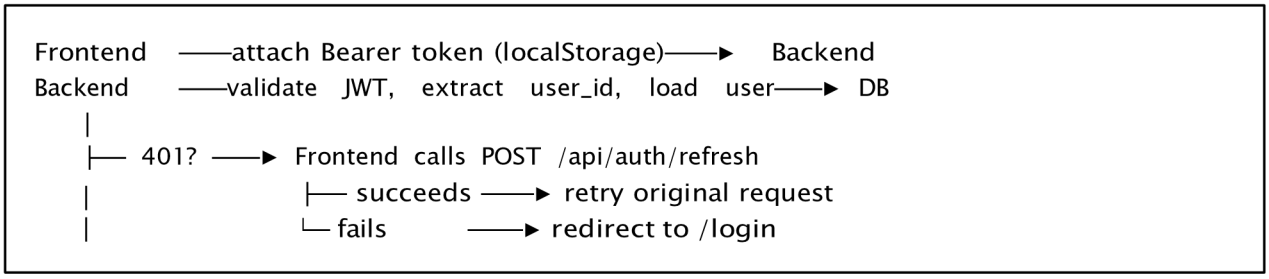


Figure 3.4. *Authentication and token-refresh sequence.*

3.9 Audio Evaluation Pipeline

3.9.1 Pipeline overview

The Evaluator (backend/audio_engine/evaluator.py) takes a recorded chord attempt, along with the expected note set for the target chord and returns a score, the identified note set and mismatching note sets, and a class of failure. Figure 3.2 provides an illustration of the implemented stage pipeline.

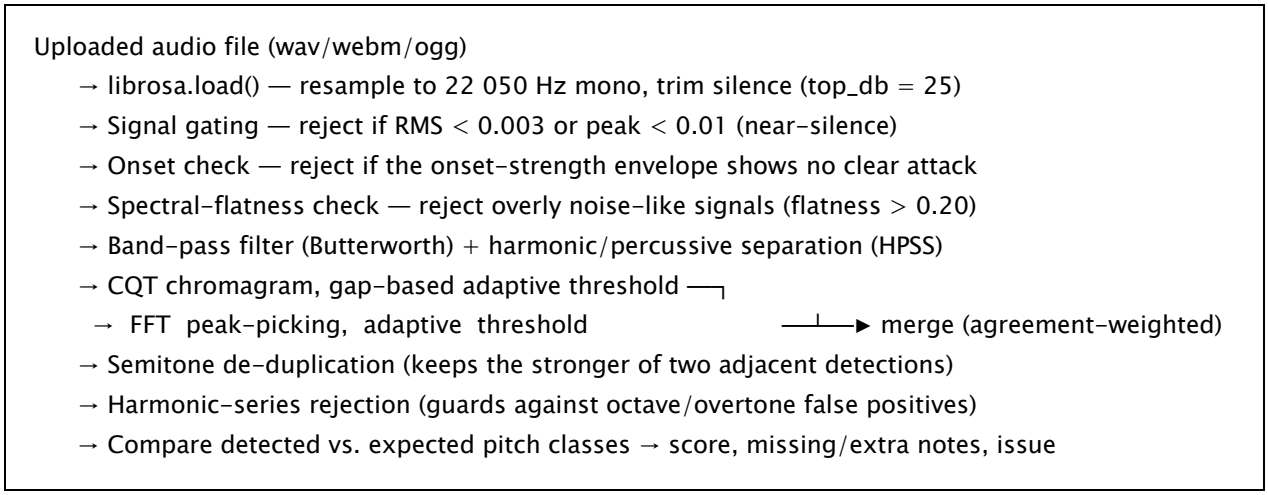


Figure 3.2. *Audio evaluation pipeline, as implemented in ChordEvaluator.*

3.9.2 Dual detection: chromagram and FFT

Interestingly, rather than settle for one type of pitch-class detection, the evaluator runs two different ones side-by-side and compares them-more work than was entailed in picking one and running with it. One detector is a Constant-Q chromagram, calculated using librosa.feature.chroma_cqt, fed from the "harmonic-emphasized" sound, and aggregated into twelve mean energies for each pitch-class. In this case, it does not adopt some absolute threshold determining whether a pitch is "on."

Instead, because recordings can differ quite a bit, it sorts the twelve energies and finds the maximum gap between energies 3 through 11 and places its criterion exactly midway of that gap,

although the minimum is kept at 0.2 and the maximum is 0.5, so it doesn't go to zero, or climb very high indeed.

This threshold is therefore adaptive to the actual degree to which any given song's pitch-class energy is peaky (contrasts sharply with the noise floor), something real songs tend not to do evenly. A second detector works altogether: an FFT-based peak-picker, found using scipy.signal.find_peaks in the 75 Hz – 1.2 kHz band, which is sure to cover conventional guitar tuning ranges. These spectral peaks are then mapped to the nearest note name. Only those notes that are confirmed by both detectors are returned as output.

That exception that if the stronger of the two is sufficiently confident even if only one, i.e., if chroma strength is greater than 0.35, or the amplitude of a peak in the FFT above 0.28.

These thresholds are a compromise since sometimes one detector will miss something that the other will detect easily.

3.9.3 De-duplication and false-positive rejection

Once the two detectors have done their work, there are two further passes run over the results, cleaning up some likely artefacts that tend to arise from detection on natural recordings rather than sterile synthetic ones. The first passes semitone deduplication: if a pair of notes turn up less than a semitone apart the smaller of them is simply discarded, as it's common enough to have spectral leakage in recordings for the other note and the FFT or chorogram to mistread it. That done we use harmonic-series rejection: this attempts to catch what look like different notes, which is instead multiple tones read from the overtone series of a single played note, by checking for patterns where three or more detected notes sit at multiples of an lowest note's frequency that approximates an integer relationship.

3.9.4 Scoring

The scoring rule that was delivered to production is, luckily, much simpler. It compares sets of the produced pitch-classes against what was expected: raw score = number of matched notes / number of expected notes. It then reduces score by 0.08 penalty per extra (unwanted) note produced, floor(at zero). Score = max(0, raw score - 0.08 extra notes).
This deserves a honest disclosure, since it is much simpler than what was originally planned for this system; see, for example, the original design spec, docs/audio-algorithm.md, which calls for a three-term, weighted scoring mechanism: 50% note-accuracy, 30% string-activation, 20% signalquality, threshold=0.80. However, concerns about string-activation and signal-quality were incorporated into the earlier gating/rejection logic previously-discussed, and were not deemed redundant to score for twice. The simplified rule prevailed.
I chose to err on the side of full and explicit disclosure, so I'm explaining the mismatch up front, in lieu of allowing the design document to be implicitly in sync with the behavior implemented in evaluator.py. All decisions made in evaluating present behavior are based on, and thus reflect, evaluator.py -- that document serves as the single source of truth about the system's behavior, not the design doc, here.
The missing/extra note pattern is categorized as one of: 'nosound', 'wrongchord', 'mutedstring', 'extranotes', or 'missing_note.' This, as explained below, directly informs the Feedback Engine to construct an actionable tip-a missing C-note on the low E may lead to a tip specific to the middlethree fret, first position E-string position of that C.

3.10 Council: AI Lesson Generation Pipeline

The Council is invoked when a user searches for a song by name. Figure 3.3 shows the pipeline from query to a fully-embedded lesson document.

POST /api/council/generate

- ingestion.py: ingest_song(query) – one Groq call extracts key, time signature, tempo/feel, difficulty, chords[], song_sections[], techniques[]
- agents.py: run_council(song) – 4 focused Groq calls, run as 2 pairs 4s apart (Theory · Technique · Ear Training · Practice Planner), each capped at 200–250 words
- chairman.py: synthesise(song, agent_output) – 1 Groq call writes the chairman_summary + quiz questions; resolves chord symbols – chord_key
- router.py: embed_fingerings_and_functions(lesson, chords) looks up each chord_key in fingerings.json, embeds FingeringPosition[] + Roman-numeral chord_function on every practice chord
- LessonDocument returned to the frontend

Figure 3.3. *Council lesson-generation pipeline: one ingestion call, four parallel agent calls run as two rate-limited pairs, and one chairman synthesis call.*

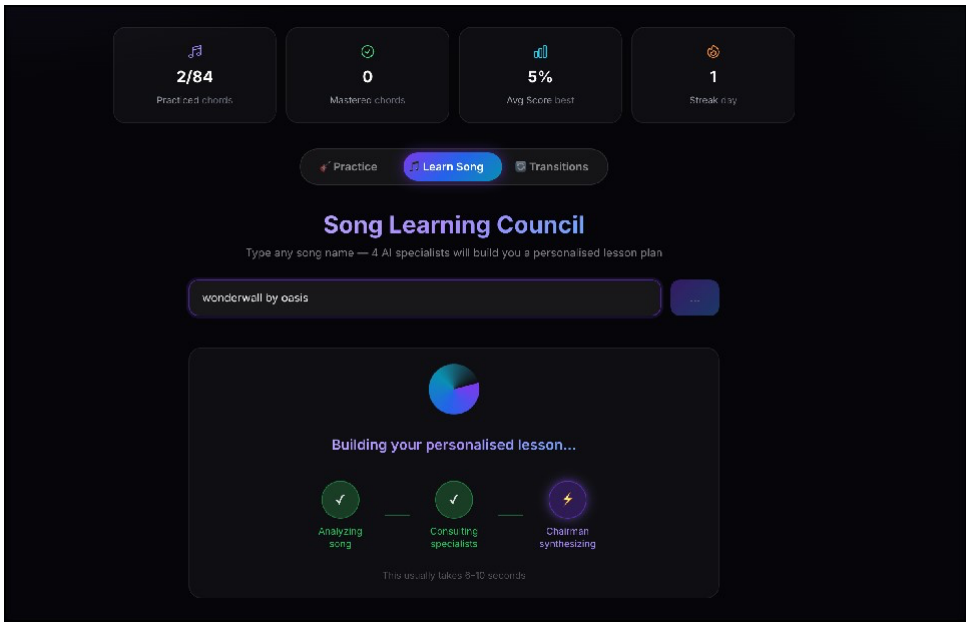


Figure 3.3b. *The three-stage progress indicator shown to the user while the pipeline in Figure 3.3 runs, typically completing in 6–10 seconds.*

Each of the 4 agents has only been deliberately handed the input fields relevant to its specific lesson slice, the ones that we laid out earlier in section 2.5. For example, the ear-training agent, has never once seen a list of which techniques were asked for, and in contrast the technique agent hasn't seen a shred of the harmonic or theory related slice of input. It is a form of deliberate information hiding really, and the point of it was to keep the prompt small so that each output could also be small without the agents treading in regions that they weren't supposed to.

The order of execution is kind of weird too, and it deserves to be explained why. Instead of sending out all four requests at once, as you would probably immediately think, the system sends out two requests at a time, spaced four seconds apart. Theory and technique are sent together first, then there's a four second gap, and ear-training and practice plan are sent together. This isn't just about elegance, in fact it is quite literally and ungracefully about accommodating the rate limit we ran into while building with the Groq free tier, capped at 15 requests a minute. By spacing things out like this, the whole generation can fit within this limit, and still be pretty fast because we're at least not waiting for 4 round trips instead of just 2.

I also want to point out that we put in one small safety net. Any given agent gets tried again (retried) if they get an HTTP 429 rate limit error response back from the Groq API. If the retried call also fails for some reason, then we fall back to a generic string placeholder. This is helpful because this means a single call hitting rate limiting should only fail one portion of the lesson, not tank the entire generation process, which is a pretty critical distinction to build into the early product life cycle.

What we get back on the other end is a LessonDocument, and it is a pretty rich structure by design. It holds song metadata, Chairman-produced summarized speech, all four pieces of section text, a practice plan string, and a list of practice chord strings (each already having its fingering pattern and its intended chord function (as a roman numeral) pre-embedded within), among other fields like the song's sectional chord structure and set of quiz questions. Ultimately, almost anything the front-end needs within the Integrated Lesson View is packed and sent back in one response, thereby avoiding further round trips in the middle of a lesson that could have been combined.

3.11 API Design

The API is a standard resource oriented REST interface with routes segmented by router prefix (/api/auth, /api/council, /api/progress, /api/plan) plus a handful of unauthenticated "core" routes for presenting chord data and handling audio uploads. We present the API routes as implemented by the system in Tables 3.5 and 3.6, with a full reference implementation table provided in Appendix C.

Method & Path	Description
GET /chords, /lesson/{chord}, /fingering/{chord}	Chord catalogue and lesson metadata
POST /learn_chord, POST /submit_audio	Start a chord session; submit an audio attempt for scoring
POST /api/auth/signup, /login, /refresh	Account creation, login, and token refresh
GET/PATCH /api/auth/me	Current user profile; update display name/skill level

Table 3.5. *Core and authentication endpoints.*

3.12 Deployment Workflow

Minor correction needed: The two services don't actually deploy the same way, and it’s a bit messy right now, so it makes sense to highlight that what follows below no longer fully applies. The frontend still auto-deploys off of the same main branch on github as it has before: anytime I push, Vercel sees the change and automatically rebuilds the site, and that process is completely unchanged. The backend's another story now however: while that one used to build automatically from Render too, it now lives over on Cloud Run, and the deploy has become more manual.

My push no longer automatically does a thing on the backend.

Instead, deploying requires typing in the following two commands in order: gcloud builds submit -config infra/cloudbuild.yaml . -this builds the image using Cloud Build-and then, assuming the above is successful, gcloud run deploy guitar-tutor-backend --image ... --region us-central1 --project ai-guitar-tutor-prod. The actual Cloud Build configuration is stored in a file called infra/cloudbuild.yaml -no longer on render.yaml, which was specific to the old Render arrangement, and now the project also includes .gcloudignore so as not to include unnecessary data in the cloud build context. Each service has their own platform-specific build file, namely render.yaml for the backend, and frontend/vercel.json for the frontend; there are also dockerfiles on both frontend/and backend/, just in case you require container builds instead of using the native builds. The root of the repository is home to docker-compose.yaml, which spins up both backendand frontenddev servers in tandem, preventing the need to start up and then manage two separate servers independently every time you wanna have an end to end session with test Data.

3.13 Testing Strategy

The repository includes an automated test suite on both sides of the stack, listed in Table 3.7.

Layer	Test files	Focus
Backend (pytest)	test_evaluator.py, test_audio_engine.py	Chord evaluator scoring and note detection
Backend (pytest)	test_api.py, test_schemas.py, test_session.py	API contracts, Pydantic schemas, session lifecycle
Backend (pytest)	test_feedback_engine.py, test_lesson_service.py, test_tutor_agent.py	Feedback generation, lesson loading, state machine
Frontend (vitest)	noteDetection.test.ts	Client-side note-frequency helper logic
Frontend (vitest)	playAlongMath.test.ts, transitionHistory.test.ts	Play-along timing math; transition-drill history helpers

Table 3.7. *Automated test coverage (source: backend/tests/, backend/test_*.py, frontend/src/lib/_tests_/).*

CHAPTER 4

Implementation and Result Analysis

This chapter takes the application screen by screen through the deployed version, essentially a walk through what the process looks like, both on the client side how each part has been implemented, and, when it make sense, how the decisions came about over potential other approaches. I am intending for it to sound a bit like, oh, you know, a conducted tour rather than just a dry explanation of what can be achieved by way of functionality. A word to the wise in passing - the images within the text, all throughout this chapter are only representations that need to be swapped out in time with the real screen captures from the application once it is properly working on live. I have placed these markers in the text so that it should be clearly visible where those images should be located later on.

4.1 Landing and Chord Practice Interface

Which, inevitably, means the homepage route, /, is actually a do-It-all: both the marketing landing page for the uninitiated visiting fresh from the wild, AND the actual chord-practice room and all the attendant history that sits at precisely the same address. And absolutely no login either, when you aren't logged in you can practice and just see results for your, uh, current session - it gets persisted nothing, absolutely nothing at all . A ChordSelector is embedded at the page, complete with pretty much every possible chord lesson available (of which there are eightyfour indeed).

Search and sort by learning-curve it remains for novice to beginner to beginner of new skills to sort out which Barre Chords you will spend 3 weeks trying to nail down.

A chord selected there sends a POST request, to, appropriately, /learn_chord. Which will begin a session for a Tutor Agent and puts the Agent's State Machine in TEACHING .

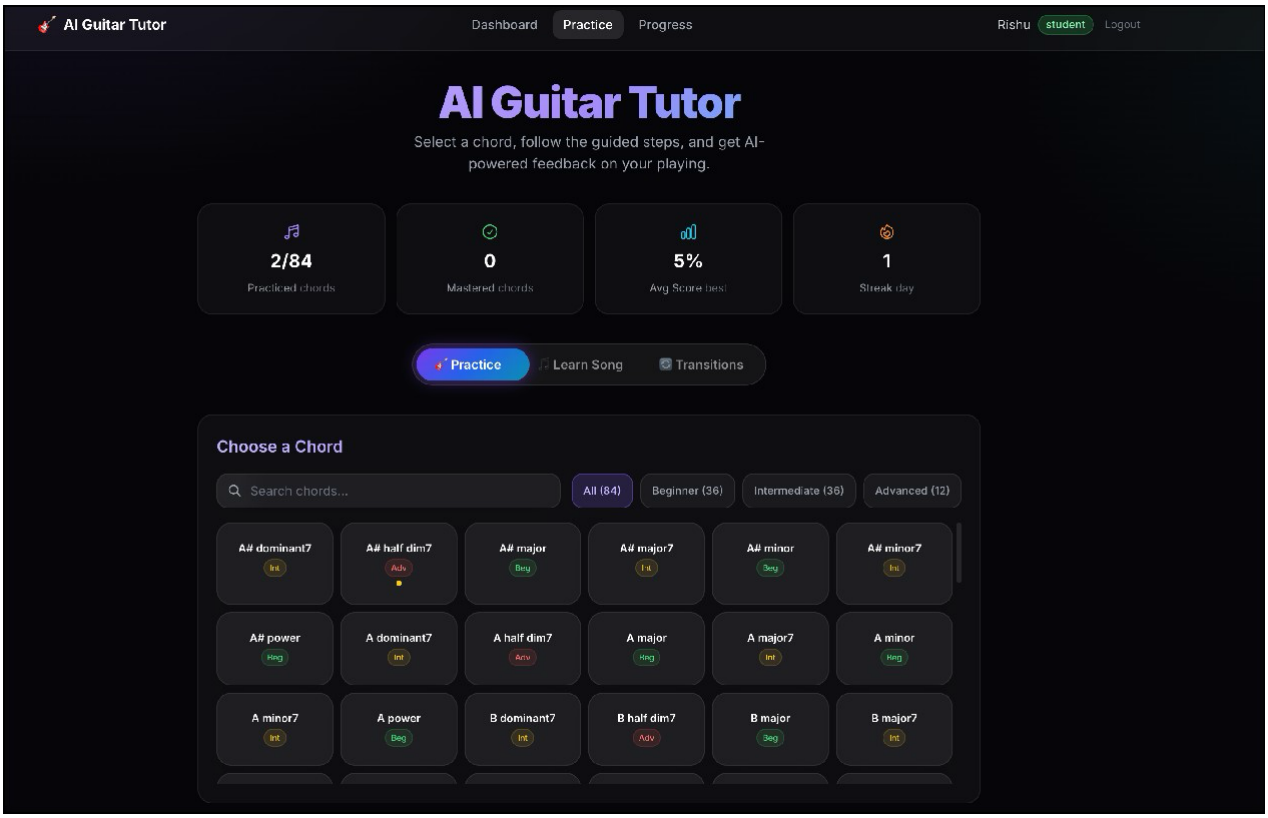
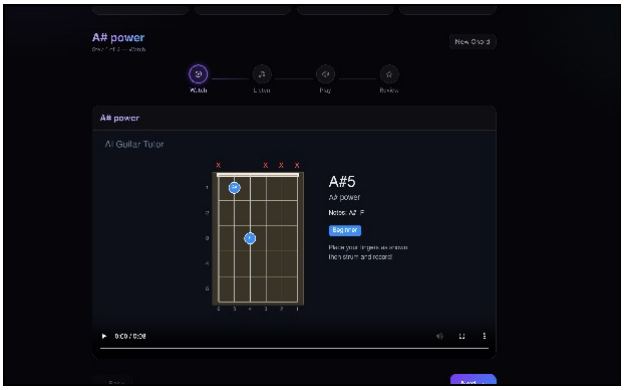
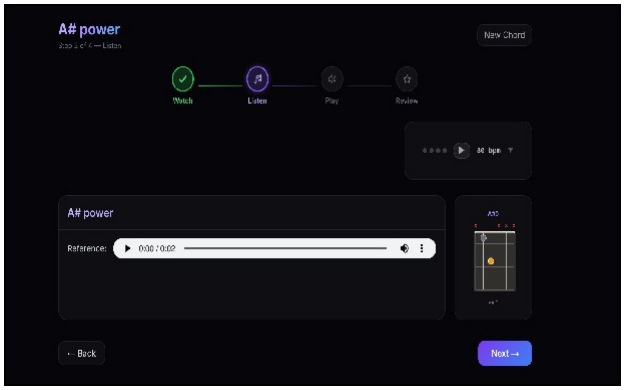


Figure 4.1. *Landing page and chord selector: the practiced/mastered/average-score/streak summary above the 84-chord catalogue, filterable by Beginner/Intermediate/Advanced.*

Once you choose a chord, the actual practice flow actually walks through four different screens sequentially:Watch, Listen, Play, Review. Quite logical naming, I think, by design. Watch involves a short example video clip, accompanied by a static SV G diagram of your chosen chord's finger placement underneath it, allowing you to watch its motion and observe its position concurrently. Listen cuts it down to just the audio to hear the optimal result of the chord before trying on your own. Finally Play allows you to get involved. Microphone Recorder, for that stage of the practice process, attempts to ask the user for permission to record a snippet of themselves playing with a metronome of adjustable pace, thus you aren't strumming at any and every whim of the user. When you click 'submit' on that audio recording and have made that a POST call on (/submit audio), you 'II find your output at the API evaluated and 'given' to you in the form of the ' score '. Along with ' score ', your detected ' notes' ' ' are provided inaddition'to missing notes and the overblown', notes as well, provided ' extra notes ' are in possession in your recording that do 'nt feature in the desired chord , they are listed under ' overblown ' too . However; ‘ ' 'when it concerns' 'missing 'notes, detailed guidance is presented which addresses each 'string','in,'terms ' of which 'finger ' ' to utilize in conjunction to the corresponding 'string. '





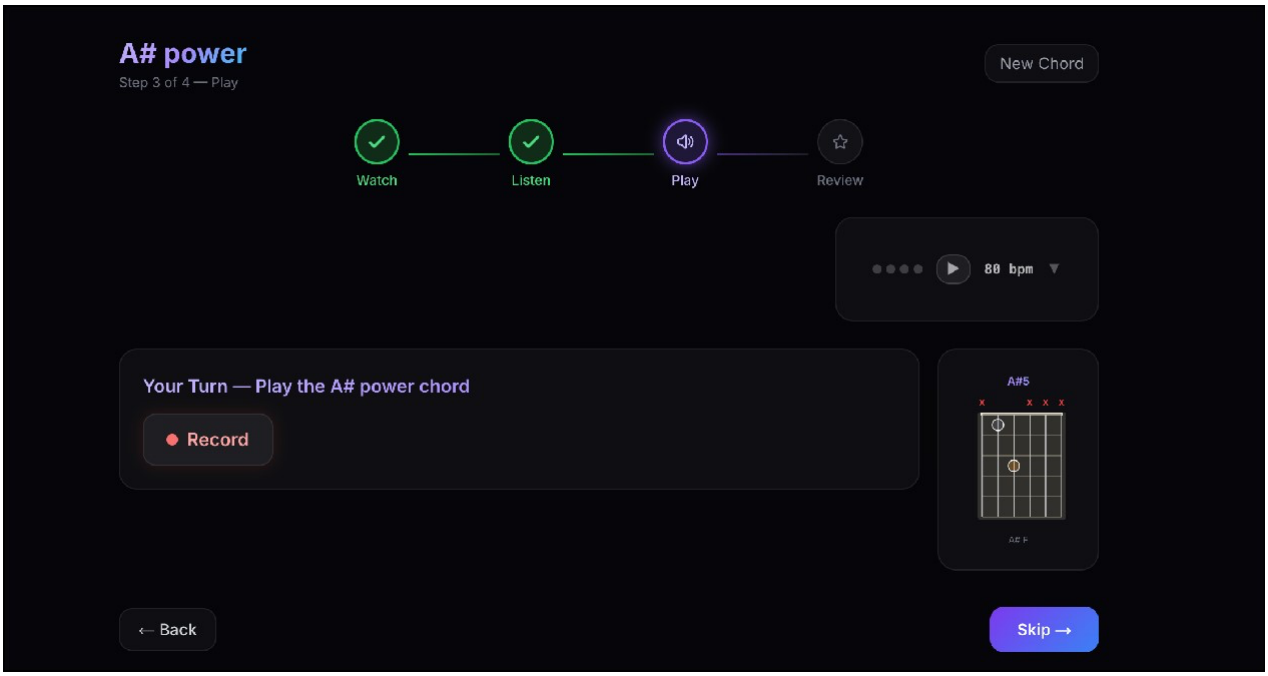
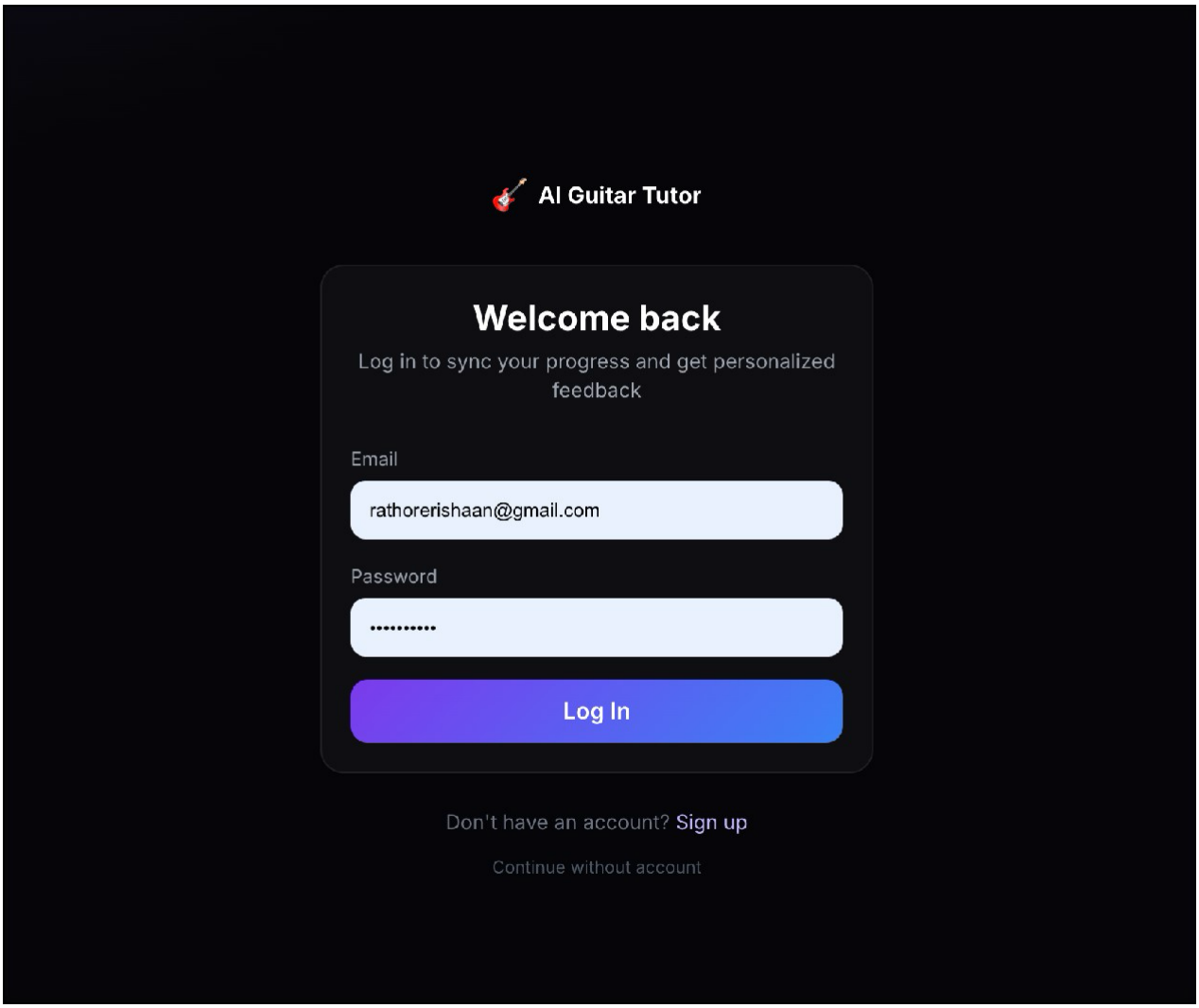


Figure 4.2. *The four-step guided practice flow for the A# power chord — Watch (reference video and fretboard diagram), Listen (isolated reference audio), and Play (Record control, 80 bpm metronome, and the SVG fretboard diagram) — leading into Review.*

4.2 Authentication Screens

The "/login and /signup pages are… both fairly unremarkable Next.js pages…" so nothing too unexpected here. "Both will naturally not be authenticated because that is the whole purpose" of such pages. They take your credentials and ship them off to the API to be consumed by the /api/auth/login and /api/auth/signup endpoints respectively. Assuming that request is successful, it'll store you with an access token, a refresh token and then push the necessary information into AuthContext, which'll unblock the more restricted dashboard/progress routes.
"The forms are basic, simple, clientside forms with inline type-ahead input validation, no lengthy multI-step wizards walking you through creating an account across several screens." This felt like a design choice. It more "in-line" with how the app itself wants to present itself and keeps it from "being too complex; who wants to spend five minutes creating a free to usechord practice tool." One last thing, "I also have a 'continue without account' for both of these pages so you can drop straight to the un-authenticated drill flow…" thus meaning an account is entirely optional.


AI Guitar Tutor
Welcome back
Log in to sync your progress and get personalized feedback
Email
rathorerishaan@gmail.com
Password
Log In
Don't have an account? Sign up
Continue without account

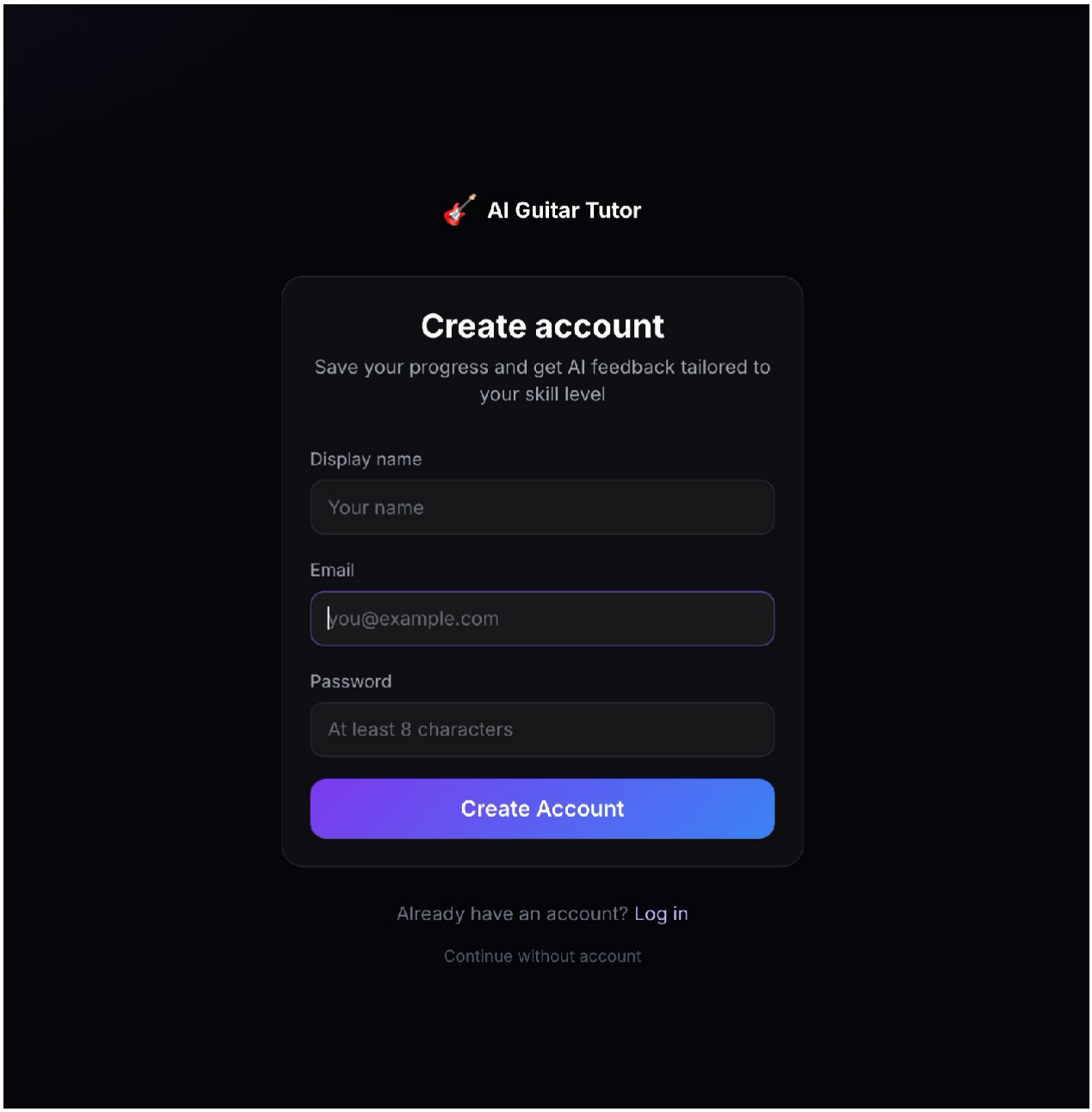


Figure 4.4. *Sign-up screen.*

4.3 Song Council and the Integrated Lesson View

So as not to distract us, SongSearchBar is just a simple free-text input field; fill this in, press return and submit, the code makes a POST /api/council/generate call, the starting of that full Council pipeline already discussed back in Section 3.10. From the back comes what feeds in the IntegratedLessonView which, frankly, is the actual raison d'être of this entire feature. As one single, scrollable surface, it contains at the top the Chairman's summarized reflection; its various key/tempo/difficulty tags in line with the text; a sectional SongTimeline that breaks the song into Intro, Verse, Chorus, Bridge and Outro, and what I have called a ChordMasteryBoard – effectively a grid detailing the specific chords for the given song and what kind of practice those accords will have.

Clicking any of these chord practice cards reveals a ChordPracticeDrawer, using the exact same record and report process that we've examined elsewhere in this document (in Section 4.1) except this time, we embed it in the song's lesson, rather than a separate lesson just of that single chord.

Each time you get any of these accords "practised" within the lesson, we will also send off another individual request; POST /api/council/practice/tip, asking for our individual prompt-toLLM tip, sitting on top of our normal, deterministically scored practice lesson.

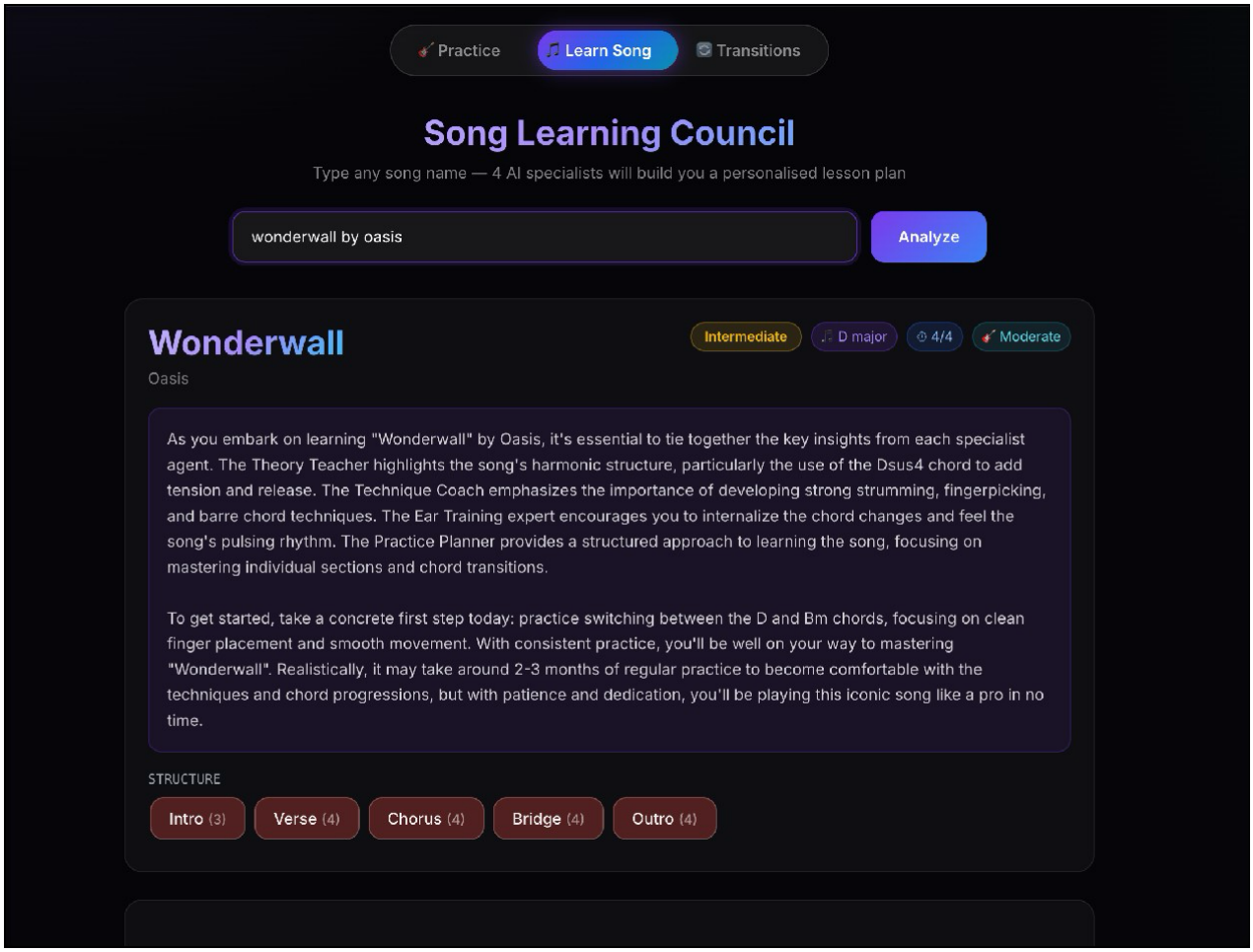


Figure 4.5. *Song search and the chairman-synthesised lesson summary for "Wonderwall" by Oasis, with key/timesignature/difficulty tags and the Intro–Verse–Chorus–Bridge–Outro structure shown below.*

Once each in-lesson practice chord has at least been touched once, the frontend can choose to invoke a call to POST /api/council/practice/revise, potentially revise the practice plan if it feels like it might need to given how well the player performed. This step is optional though, it's not a necessary feature in a lesson. Afterward, LessonQuiz presents a multiple-choice quiz from POST /api/council/quiz, if a chord was failed on it does link directly back to its associated card so you have a chance to get it correct without allowing negative knowledge to fall thru the cracks to manifest in a more expensive context, passing then automatically enables PLAY_ALONG mode. Which consists of a chord-progression player set to the in-lesson section arrangement.

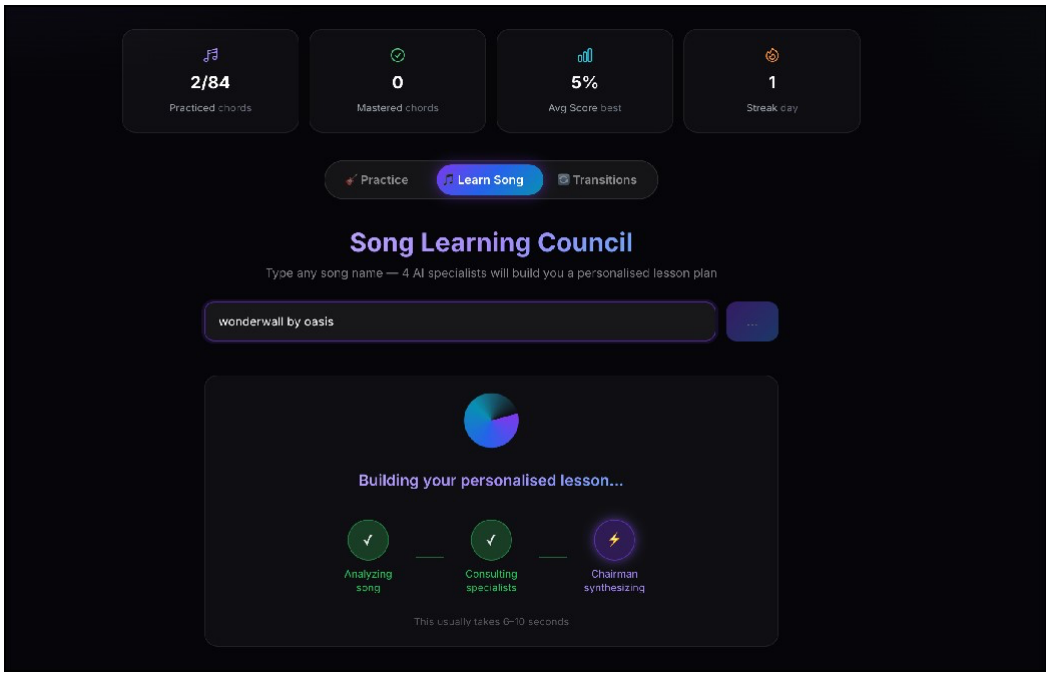


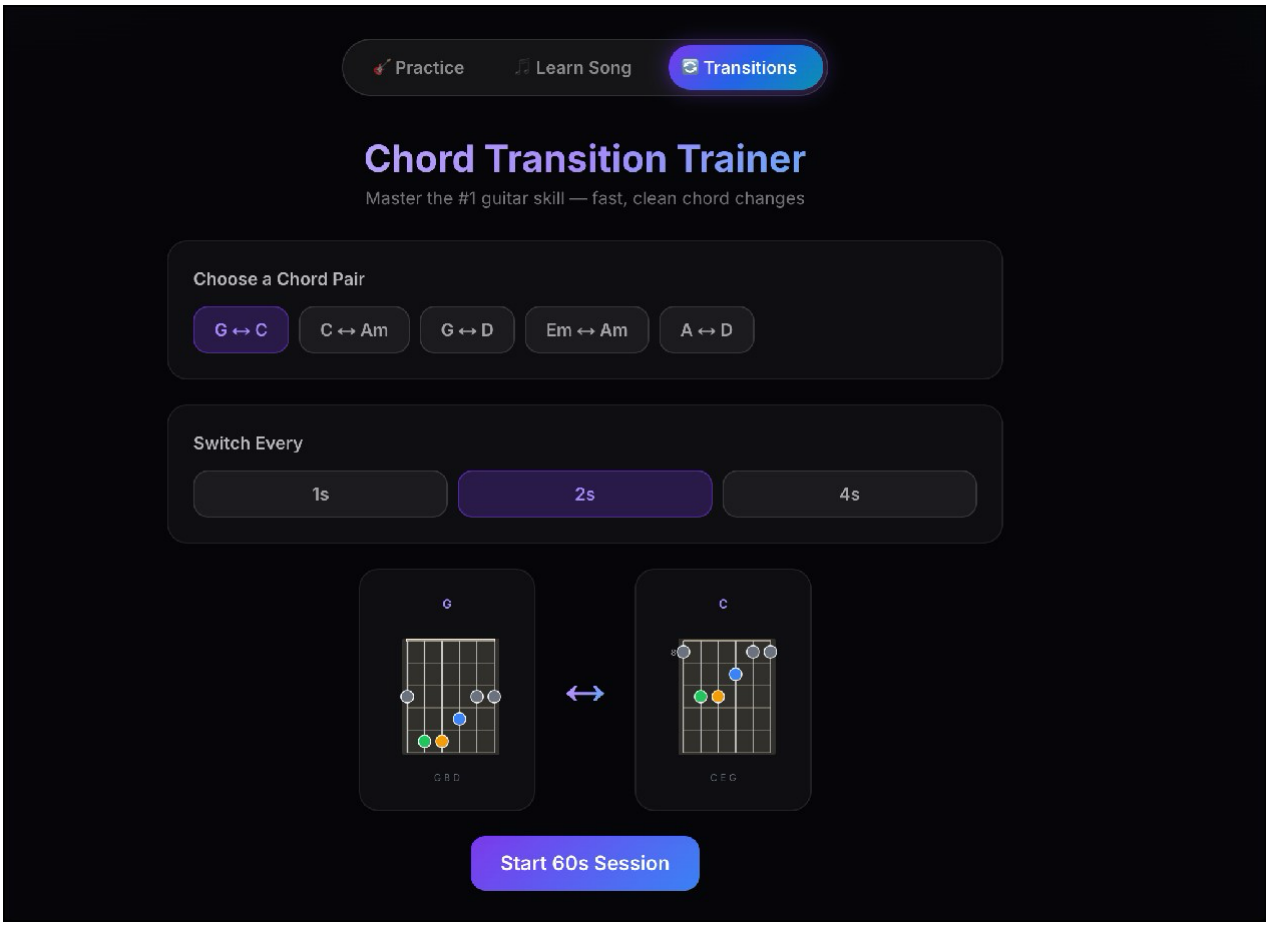
Figure 4.6. *The Council's generation state shown while a lesson is being built (analysing song → consulting specialists → chairman synthesising), typically completing in 6–10 seconds.*

4·4
Transition Trainer

TransitionTrainer is quite a different flavor of drill to anything else we've looked at previously. It's paced, timed to a metronome, and it cycles a prompt between two you've selected chords, while measuring how accurately, and how quickly you transition between them. That rate then manifests itself as transitions per minute (TPM), calculated completely client-side via lib/useMetronome.ts and - again, for logged-in users only - saved via a POST to /api/progress/transitions.

There's no other way, from a functional perspective, to compare that against the standard chord practice flow other than carving it out on its own and say; it's a real measure of a different thing.

The audio evaluator we looked at before just said whether or not, in that instance, you hit the "correct" chord, whether full or empty - it has nothing to tell us about what your rate between chords mid-song looks like, which as any student who's attempted to pivot between C and F during a verse in a timely fashion without grinding to a halt will know is very nearly a unique talent of its own. That's where TransitionTrainer has been born from.



4·5

Figure 4.7. *Transition trainer drill: chord-pair selection (G↔C), switch-interval control (1s/2s/4s), and the two chord fretboard diagrams being alternated.*

Dashboard and Personalised Daily Plan

The /dashboard is home to the app's more personalized side. Displayed is Today'sPlan, grabbed over SWR from GET /api/plan/next, as well as QuickStats and a 52-week StreakHeatmap – honestly, after a few weeks you kind of look forward to that particular visual. Today'sPlan is pieced together by personalization/planner.py and built solely around the student's own history, chords attempted, and transition drills, never from someone else's information.

It's not laid out casually, either; the structure contains a selection of chords you've recently succeeded with as warmup (ease into the session), your focus chord - whicheverchord has recently ranked the lowest - the one most benefiting from extra-study time - a new challenge chord (pulled from the lowest unimplemented skill tier), and, lastly, a transition practice composed from whichever pair you most often miss.

The plan, incidentally, isn't ever more than five items for beginners and up to seven items for other, more practiced, students, to avoid any particular lesson feeling like homework.

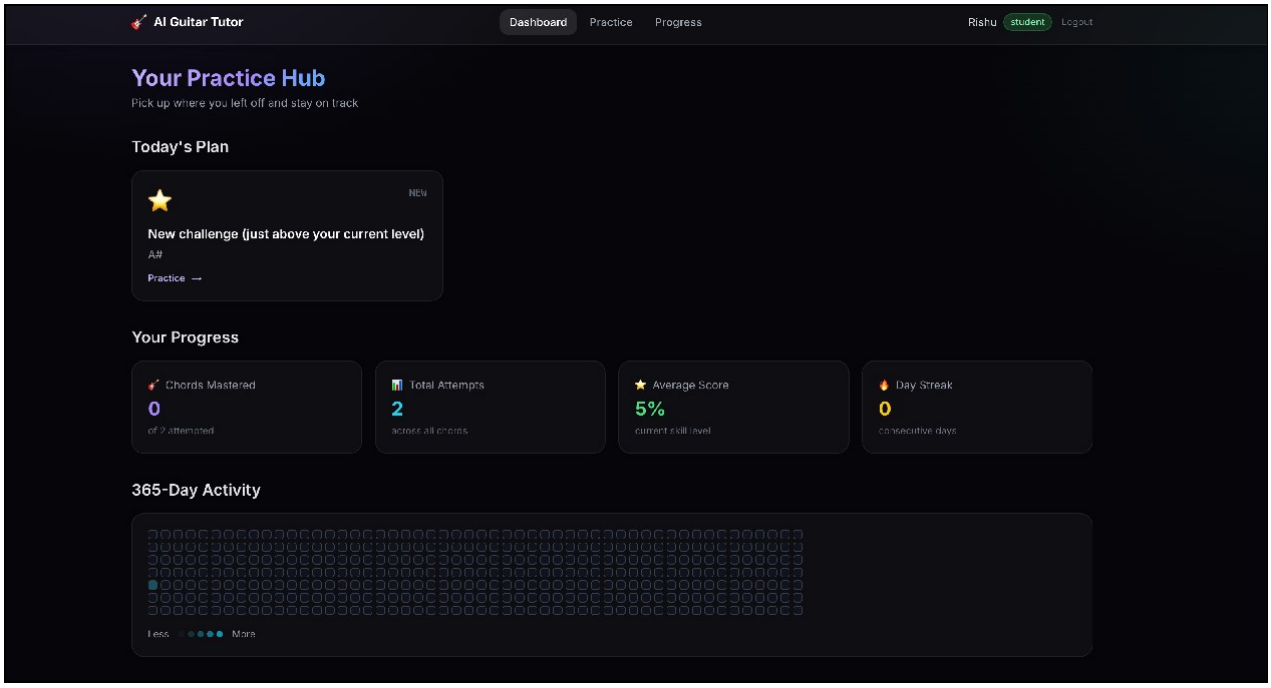


Figure 4.8. *Dashboard: "Today's Plan" new-challenge card above the Chords Mastered / Total Attempts / Average Score / Day Streak summary and the 365-day activity grid.*

Progress Analytics

4.6

This rendering will output a CalendarHeatmap pulled from GET /api/progress/calendar, alongside a Beginner/Intermediate/Advanced meter that will directly correspond to a student's saved attempts History-nothing behind it but what's there-no complex modeling just an aggregation of whatever history the student has built.

If you want even more depth than that, then you can navigate to that specific note, /progress/chord/[name] that you can access from the chord mastery grid, the individual mastery data pulls a complete attempt timeline, GET /api/progress/chord/{name}/history. Now rather than taking just one arbitrary mastery on faith you can click that data and then actually look at all of the attempts you've made at, for instance, this F chord, step by step, see your practice attempt by attempt how you went (and hopefully haven't degraded):

.

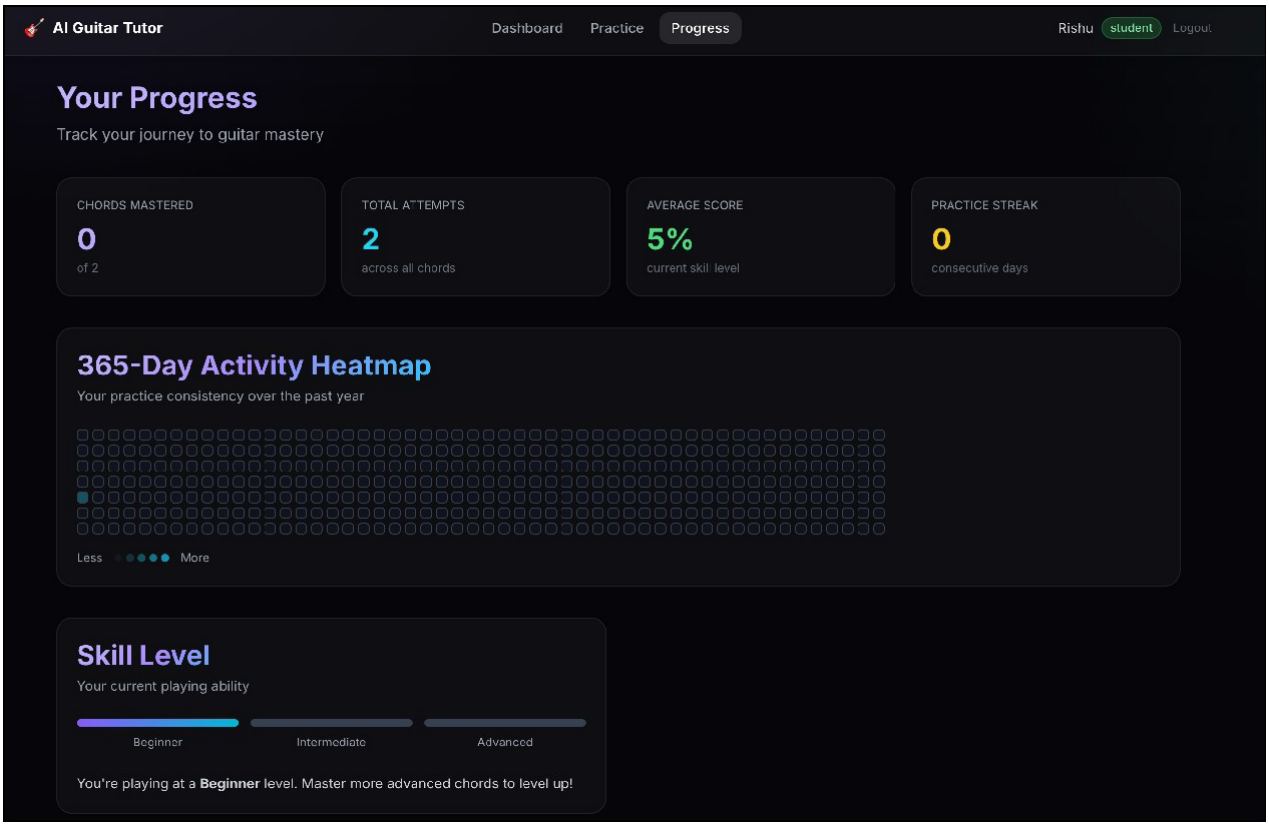


Figure 4.9. *Progress dashboard: chords-mastered/attempts/average-score/streak summary, the 52-week activity heatmap, and the Beginner/Intermediate/Advanced skill-level meter.*

Result Discussion and Limitations

Screen / Feature	Primary backend module(s)
Chord practice + feedback	audio_engine, feedback_engine, tutor_agent

4·7

Song lesson generation	council/ (ingestion, agents, chairman, router)
Dashboard / daily plan	personalization/planner.py
Progress analytics	progress/router.py
Teacher / Admin views	teacher/, admin/

Table 4.1. *Screen-to-module mapping.*

Once I zoom out, and survey the scope of this chapter alongside the totality of Chapter 3, two things emerge in clear relief: 1. I think the dual-detector audio evaluation engine with its gap-based adaptive threshold really does manage to adjust to various recording conditions in a way that a single fixed-threshold detector wouldn't have. So that aspect seems to work out.

But let me be honest what it still doesn't do.

It doesn't provide on-the-fly or continuous feedback, such as feedback as a chord rings out halfway through a strum as a live lesson-giver across from me might provide. It doesn't even evaluate, " Hey, let me listen to that. Try the next three notes exactly like that. Let me see if that's good."

It provides a discrete, per-attempt evaluation, and that alone is probably the biggest compromise the lesson design has to make - it impacts the flow enormously, it makes the practice feel a bit clumsy and artificial."

2. The two-pairs-with-a-gap for the Council's run order represents an actual workaround with a direct consequence - I was using that not because it was in itself something like a novel or useful concept for parallel work, but because if I tried to have all four tasks running together against the freetier that the Coursera team would be able to evaluate me, the Coursera team will try to see how hard that's going to work and probably throw an error message. Hence that's the Council's workaround that came about not out of design consideration - or at least not from design consideration alone - but just because we were able to get the work around to come up. All4 agents will definitely execute faster if they're in paralle; for free, the answer is to separate the groups with a gap.

Item from the original design brief	Status in this implementation
Chord practice with audio scoring	Implemented (Section 3.9)

Song lesson generation (4-agent council)	Implemented (Section 3.10)
SVG fretboard, transition trainer, daily plan, progress dashboard, quiz	Implemented
Teacher dashboard, admin panel	Implemented
CREPE neural pitch detection	Listed only as a future improvement in docs/audioalgorithm.md; not present in evaluator.py
Continuous real-time chord evaluation during play	Not implemented — evaluation is per discrete recorded attempt

Table 4.2. *Implemented vs. documented-only capabilities, cross-checked against the repository's own design notes.*

CHAPTER 5

Conclusion

5.1 Summary

AI Guitar Tutor at the end of the day is a deployed, full-stack web app that combines a signalprocessing chord-evaluation algorithm with a multi- agent, llm-generated lesson creation engine, inside of a role-based ui that combines practice, tracking and teaching under a single Roof; The Next 14 Next App running on vercel talks exclusively to a FastAPI service on render that backs onto PostgreSQL in separate and independent deploys that talk exclusively over a documented rest api

5.2 Achievements

A workingdual chromagram / FFT chord evaluator (adaptivesemifrequentgaps,semitone deduplication,harmonic-serialfalse positives rejection) to84 chordsimplemented. Afour agentwithChairman agentAI lesson pipelinethat staysextrafree LLM price cap yet generates a organized, chord-linked, lesson with quiz endings out of 1-input song-name query. Auser friendlylayer generates the daily exercises list solely from previous learner history, 0.o, separately trained model.

Security Layer usingJWT JWT access and renew tokens combined withpasswordprotectEDrenewtoken storagethat secured any learner data.

A suite automatically-run testing that ranges across theevaluator,ApiContracts,session logic, and important user side features (detecting the notes, transition history, and playAlongtiming) that would help check performance and expose issues immediately after they may occur.

5.3 Limitations

Chord detection is done per recorded take/instance not per slice on a real-time play. CREPE model for pitch is not used; classical chromatic (fft) approach is fully used in order to remain computationally cheapest, this path was envisioned in the design document as a possible, but not essential path. Council’s double two’s-an-opening for rate limitation rather than optimal lowlatency approach. Lesson generation uses prior knowledge from the base LM on a song, as it does not analyse uploaded recordings.

5.4 Future Scope

The repository's design notes themselves indicate about a dozen lines that would fit reasonably well in the system's architecture without a complete rethink of the entire stack. A deep learning pitchdetection algorithm, presumably CREPE, could either work alongside the current twin pitch detector or even (after further work) become the main pitch detector replacing some if not all of it. A fully-fledged chord-detection model not relying on template matching but learning directly from audio recordings could likely deal with corner cases more effectively. Continuous and live feedback on player action rather than discrete test-before-and--after scores are other promising avenues to investigate - perhaps the most valuable, but likely hardest one. Finally, noise rejection will undoubtedly need more attention to cater to students using poor in-app or built-in microphones rather than specialist hardware.
Some of these are touched upon in the design documents, while others are further from these; nevertheless they represent potential future areas: A mobile version or PWA could comfortably utilize the system, as it would merely consume the underlying REST API with minimal modification and, ideally, transition to pay per query (PPQ) or alternative, cost-efficient provider such as Bard which would mean the system could de-prioritise "pairing" individual LLM agents and instead effectively leverage 4 independent LLMs, cutting down generation times.

5.5 Final Conclusion

What I hope this project has shown finally is that the two should not exist in different tools, but rather live side-by-side inside of one system: a deterministic signal-processing signal-evaluation engine paired with a generative LLM content engine - so long as you be meticulous about designating which tool is which (the signal-processing pipeline gets ground-truth privileges while LLM will not). The output-for the moment at least-is a deployed and working system for teaching beginning guitarplayers, by giving them simultaneously: the brutally honest answer "was my last chord in tune?', and the roadmap for ' what do i need to do to practice then?' built out of Nothing more than a bunch of open source libraries and one paid hosted LLM service, not including nothing exotic nor specialized hardware to speak of . Just a tab, the sound around us, and a guitar probably close to the first thing that springs to mind the second the tab opens the.

REFERENCES

---

1. Vercel Inc., *Next.js Documentation*. [Online]. Available: https://nextjs.org/docs
2. Sebastián Ramírez, *FastAPI Documentation*. [Online]. Available: https://fastapi.tiangolo.com
3. B. McFee et al., "librosa: Audio and Music Signal Analysis in Python," in *Proc. 14th Python in Science Conf. (SciPy 2015)*, 2015, pp. 18–24.
4. A. de Cheveigné and H. Kawahara, "YIN, a fundamental frequency estimator for speech and music," *J. Acoust. Soc. Am.*, vol. 111, no. 4, pp. 1917–1930, 2002.
5. The SciPy Community, *SciPy Signal Processing Documentation (scipy.signal)*. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/signal.html
6. SQLAlchemy Project, *SQLAlchemy 2.0 Documentation*. [Online]. Available: https://docs.sqlalchemy.org
7. Groq Inc., *GroqCloud API Documentation*. [Online]. Available: https://console.groq.com/docs
8. Meta AI, *Llama 3.3 Model Card*. [Online]. Available: https://github.com/meta-llama/llama-models

9. Meta Platforms Inc., *React Documentation*. [Online]. Available: https://react.dev

10. Tailwind Labs, *Tailwind CSS Documentation*. [Online]. Available: https://tailwindcss.com/docs

11. M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

12. Mozilla Developer Network, *MediaStream Recording API*. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/MediaStream_Recording_API

13. R. Rishaan, *ai-guitar-tutor* [Source code repository]. GitHub, 2025–2026. [Online]. Available: https://github.com/rishaan1402/ai-guitar-tutor

14. *AI Guitar Tutor* [Deployed application]. Vercel, 2026. [Online]. Available: https://aiguitar-tutor.vercel.app/

APPENDIX A — PROJECT FOLDER STRUCTURE

```
frontend/src/app/              page.tsx, dashboard/, progress/, profile/, admin/,
teacher/, login/, signup/
frontend/src/components/       SongCouncil/ (15 files), feedback/, AdminPanel/,
TeacherDashboard/, layout/, ui/ (15 files)
frontend/src/lib/              api.ts, auth.ts, motion.ts, progress.ts,
noteDetection.ts, useMetronome.ts
backend/audio_engine/          audio_io.py, evaluator.py
backend/council/               agents.py, chairman.py, ingestion.py, advisor.py,
quiz.py, session_store.py, router.py, schemas.py
backend/feedback_engine/       generator.py
backend/auth/                  router.py, security.py, dependencies.py,
profile_context.py, schemas.py
backend/db/                    engine.py, models.py
backend/progress/, personalization/, teacher/, admin/, lesson_service/
backend/tests/, backend/test_*.py      pytest suite
data/lessons/                  84 chord folders (metadata.json, reference.wav,
lesson.mp4)
data/chords/                   fingerings.json, chord_definations.json
scripts/                       extract_chords.py, generate_fingerings.py,
generate_videos.py
docs/audio-algorithm.md, .architecture.md, readme.md
docker-compose.yml, infra/cloudbuild.yaml, and .gcloudignore
```

APPENDIX B — TECHNOLOGY STACK SUMMARY

See Table 3.1, Table 3.2, and Table 3.3 in Chapter 3 for the full annotated stack.

APPENDIX C — API ENDPOINT LIST

Prefix	Endpoints
Core	GET / , GET /chords, GET /lesson/{chord}, GET /fingering/{chord}, GET /audio/{chord}, GET /video/{chord}, POST /learn_chord, POST /submit_audio, GET /session, POST /reset
/api/auth	POST /signup, POST /login, POST /refresh, GET /me, PATCH /me
/api/council	POST /generate, POST /practice/tip, POST /practice/revise, POST /quiz
/api/progress	GET /, GET /calendar, GET /chord/{name}/history, GET /transitions, POST /transitions, POST /quiz/{lesson_id}/submit
/api/plan	GET /next, GET /weekly

APPENDIX D — ENVIRONMENT VARIABLES (SECRETS EXCLUDED)

Variable	Where	Purpose
GROQ_API_KEY	Backend (Cloud Run, via Secret Manager)	Required for Council / AI lesson generation
DATABASE_URL	Backend (Cloud Run, via Secret Manager)	Postgres connection string
SECRET_KEY	Backend (Cloud Run, via Secret Manager	JWT signing key
ALLOWED_ORIGINS	Backend (Cloud Run, via Secret Manager	Comma-separated CORS allow-list

NEXT_PUBLIC_API_URL	Frontend (Vercel)	Backend base URL

APPENDIX E — CONFIGURATION FILES

---

infra/cloudbuild.yaml (Cloud Build configuration), .gcloudignore (build context exclusions)), frontend/vercel.json (Vercel build settings), backend/Dockerfile and frontend/Dockerfile, dockercompose.yml (local multi-service run), backend/alembic.ini (migration configuration), frontend/tailwind.config.ts and frontend/next.config.js.

APPENDIX F — GLOSSARY

Term	Definition
Chromagram	A 12-bin representation of audio energy folded onto musical pitch classes regardless of octave
Council	This project's four-agent-plus-chairman AI lesson generation subsystem
Onset	The moment of sudden energy increase marking the start of a played note or chord
Semitone deduplication	Discarding the weaker of two detected notes a semitone apart, to remove spectralleakage artefacts
TutorAgent	The backend's per-session state machine governing one chord-practice interaction

APPENDIX G — ABBREVIATIONS AND ACRONYMS

AI - Artificial Intelligence API -Application Programming Interface CORS - Cross-Origin Resource Sharing CQT - Constant-Q Transform DSP - Digital Signal Processing FFT - Fast Fourier Transform HCI - Human-Computer Interaction HPSS - Harmonic-Percussive Source Separation ITS - Intelligent Tutoring System JWT - JSON Web Token LLM - Large Language Model MIR - Music Information Retrieval ORM - Object-Relational Mapping REST - Representational State Transfer SWR - "Stale-While-Revalidate" (data-fetching library) TPM - Transitions Per Minute TTL - Time To Live UI/UX - User Interface / User Experience

Project Details

Student Details			
Student Name	Rishaan Rathore		
Register Number	210968029	Section / Roll No	B
Email Address	rathorerishaan@gmail.com	Phone No (M)	9918280546
Project Details			
Project Title	Agentic Tutor		
Project Duration	6 months	Date of reporting	10 /1/2026
Organization Details			
Organization Name	Muziclub Music Services Pvt. Ltd		
Full postal address with pin code	office -12, Aditi Commerce, Near Food Bazaar, Baner, Pune -411045		
Website address	www.muziclub.com		
External Guide Details			
Name of the Guide	Ms. Rupinder Kaur Chahal,		
Designation	Head of Technology		
Full contact address with pin code	NA		
Email address	NA	Phone No (M)	NA
Internal Guide Details			
Faculty Name	Soumya Nandan Mishra		
Full contact address with pin code	School of Computer Engineering, Manipal Institute of Technology, Manipal – 576 104 (Karnataka State), INDIA		
Email address	soumya.mishra@manipal.edu		

Paper

ORIGINALITY REPORT

2%	2%	0%	2%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Manipal Academy of Higher Education (MAHE) Student Paper	1%
2	www.diva-portal.org Internet Source	<1%
3	Submitted to The Open University of Hong Kong Student Paper	<1%
4	evbab.manipal.edu Internet Source	<1%
5	Submitted to Atlantic Technological University Student Paper	<1%
6	Submitted to University of Winchester Student Paper	<1%
7	repository.effatuniversity.edu.sa Internet Source	<1%
8	Submitted to Rajiv Gandhi National Institute of Youth Development Student Paper	<1%
9	www.cliffsnotes.com Internet Source	<1%
10	dspace.daffodilvarsity.edu.bd:8080 Internet Source	<1%
11	lib.buet.ac.bd:8080 Internet Source	<1%
12	pt.scribd.com Internet Source	<1%
13	ruor.uottawa.ca Internet Source	<1%

Exclude quotes	On	Exclude matches	< 3 words

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

muziclub
live music

Ref.

Date : 4/1/26

Date: 4 Jan 2026

To: Whosoever it may concern

Subject: Confirmation for Internship at Muziclub Music Services Pvt. Ltd

This is to certify that Rishaan Rathore, a B.Tech student in Data Science and Engineering, is undertaking his internship with us from 10 January 2026 to 20 June 2026 on the project titled "Agentic Tutor".

During this internship, Rishaan will develop and implement new agentic AI capabilities within the Muziclub platform to enhance the student learning experience and to enable high-impact marketing campaigns through AI-driven personalization and contextual content generation.

He will be supervised and mentored by Ms. Rupinder Kaur Chahal, Head of Technology at Muziclub.

MUZICLUB MUSIC SERVICES PVT. LTD.

Devinder Singh Chahal

Director

Muziclub Music Services Pvt. Ltd

Office # 12, Aditi Commerce, Near Food Bazaar, Baner, Pune - 411045
Email: info@muziclub.com **Web:** www.muziclub.com